TEAM SINθ

ENGINEERING JOURNAL

WRO Future Engineers 2026

Aarav Daudia • Harshad Mahajan • Rajas Salunke

Github: github.com/JumpyTruck/WRO-FE-2026-Sin-Theta

Table of Contents

Abstract 4

1.0 Team Profile 5
- 1.1 About Us 5
- 1.2 The Challenge 6

2.0 Meet The Team 7

3.0 Our Journal & Competitive Goal 7

4.0 From Nationals to Internationals 8

5.0 Finance 9
- 5.1 Bill of Materials (BOM) 9
- 5.2 Miscellaneous Mechanical Parts 10
- 5.3 Budget vs. Actual 10

6.0 Season Workflow 10

7.0 Game Analysis 11
- 7.1 Previous Competition Analysis 11
- 7.2 Strategy & Approach 12
- 7.3 Our Final Approach 12

8.0 Research & Concept Development 13

9.0 Mobility & Mechanical Design 14
- 9.1 Mechanical Requirements 14
- 9.2 Chassis 14
- 9.3 Wheels 15
- 9.4 Differential & Drivetrain 16
- 9.5 Rear Axle Casing (V3) · 9.6 Rear Axles 17
- 9.7 Steering System (Ackermann) 17
- 9.8 Electronics Plates 19
- 9.9 Robot Height & Camera Placement 19
- 9.10 Motor Adapter · 9.11 Bumper 20
- 9.12 Manufacturing & Material Specifications 20
- 9.13 Final Mechanical Assembly & Validation 21

10.0 Power & Sensor Architecture 22
- 10.1 Motor Requirements / Selection 22
- 10.2 Battery Selection 22
- 10.3 Motor Driver Selection 23
- 10.4 PCB Version 3: Final Version 23
- 10.5 Raspberry Pi 4B + Arduino Nano 25
- 10.6 Sensor 26

11.0 Software Architecture & Obstacle Strategy 26
- 11.1 Raspberry Pi + Arduino Nano 27
- 11.2 Vision Pipeline 28
- 11.3 Wall Following 29
- 11.4 Obstacle Avoidance 30
- 11.5 Crash Detection 31
- 11.6 Lap Counter 31
- 11.7 Parking 32
- 11.8 The State Machine 32
- 11.9 Arbitration 33
- 11.10 Software Testing & Iteration 34
- 11.11 Performance & Validation 35

12.0 Systems Thinking & Engineering Decisions 35
- 12.1 Trade-Offs 36
- 12.2 Constraints 36
- 12.3 Risks & Failures 37

12.4 Iterations 39
12.5 Safety, Reliability & Known Limitations 42
12.6 Testing & Validation for Reliability 43

13.0 Reproducibility & GitHub 43
13.1 How to Assemble the Robot 44
13.2 How to Build the Custom PCB 45
13.3 Software Setup 46
13.4 First Test Run on Practice Mat (30 min) 47
13.5 Future Improvements 47
13.6 GitHub Utilisation: Issues & Projects 48

Conclusion 48
References & Technical Sources 48

Abstract

ROBOT

Team SinΘ Robot (V3, International build): a from-scratch autonomous vehicle built for the WRO Future Engineers 2026 Open and Obstacle Challenges. It evolved from our V1 prototype and our V2 National robot through a documented build, test and redesign process.

KEY INNOVATION

A **single-sensor, continuous-motion architecture**. One forward-tilted Raspberry Pi Camera Module 3 Wide (no ToF, ultrasonic or gyro) drives every navigation decision through a **LAB colour-space vision pipeline**. Its colour masks are computed in parallel on four worker threads, keeping the sense-think-act loop at roughly **150 ms per frame**. A single program detects which challenge it is facing (pink parking walls in view at the start mean Obstacle Challenge) and which direction to drive. Steering is then resolved by a **priority-based arbitration layer**: outer-wall safety, then inner-wall damping, then pillar avoidance, then wall following. Speed comes from a **pillar-aware variable speed profile** that boosts to full PWM when a pillar is far away and slows step by step as it gets close.

The software runs on a Raspberry Pi 4B paired with an Arduino Nano for real-time motor and servo control. The chassis pairs an **F1-inspired long-wheelbase layout** with **100% Ackermann steering** (12° inner / ~9° outer at full lock) and a **metal differential** on a direct-drive 510 RPM motor. It also uses **stainless-steel rear axles** in a single enclosed rear axle casing and a fully modular, independently serviceable three-plate stack.

HEADLINE STATS

TOP WHEEL SPEED	SENSORS USED	PROCESSING CYCLE
0.598 m/s	**Pi Camera 3 Wide**	**~150 ms**
calculated, under load	single sensor	capture → command
DRIVING ACCURACY	**LAP TIMES**	**ROBOT WEIGHT**
~95%	**24 s / 50 s**	**1.005 kg**
15 consecutive laps	Open / Obstacle	limit 1.5 kg

KEY ACHIEVEMENT

We diagnosed and resolved two full PCB failure cycles: first a 7805 linear-regulator thermal shutdown (up to 15.6 W of dissipation), then a motor-driver/servo power-isolation failure. We redesigned the electrical architecture around a buck converter and a dedicated servo UBEC. Between Nationals and Internationals we then re-engineered the camera position, rear drivetrain, Pi enclosure and software. The result is a stable, camera-only robot that finishes the Obstacle Challenge in about 50 s instead of 70 s, at the same ~95% accuracy. The whole robot was built from scratch for **$478.87** in parts.

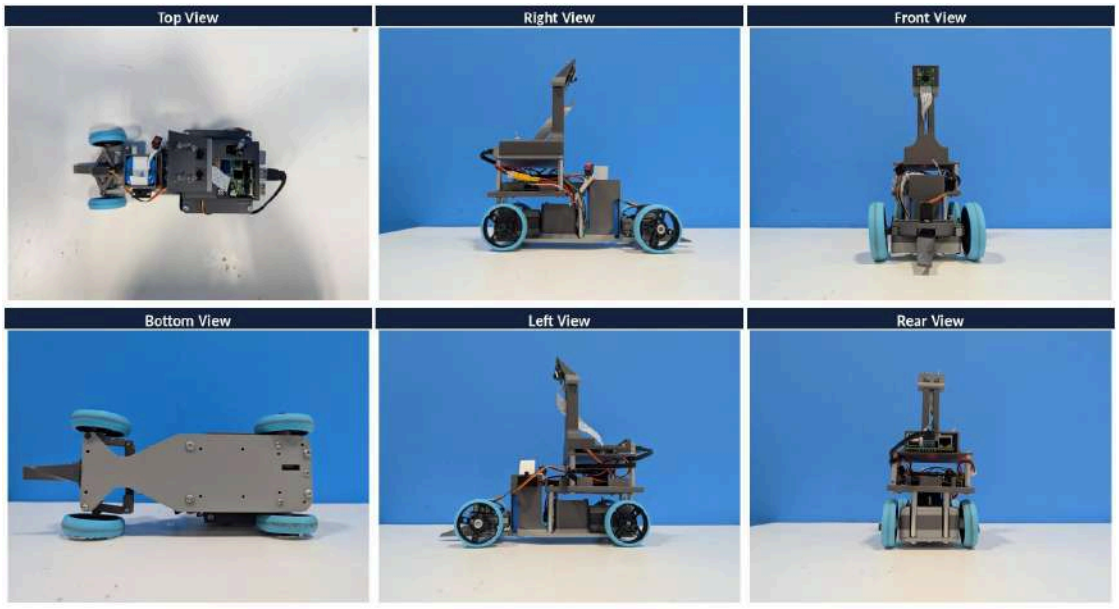


Figure A — International robot (V3): top, right, front, bottom, left and rear views

1.0 Team Profile

1.1 About Us

We are **Team Sin Theta (SinΘ)**, competing in WRO Future Engineers 2026 with three years of combined robotics experience from two previous competition teams: **The Robertos** [aka AA Batteries] (Aarav) and **VHR Motors** (Harshad, Rajas). This year we combined that experience to design a high-precision autonomous robot that puts **consistency, reliability and engineering rigour** first.

Design philosophy. Our aim was to maximise consistency rather than peak speed. Throughout development we prioritised repeatable autonomous behaviour. That led to a camera-based robot with Ackermann steering, a split Raspberry Pi and Arduino architecture, and an engineering process backed by quantitative testing, design iterations and failure analysis. Only once the robot was consistent did we start to buy back speed, which is the story of our Nationals → Internationals upgrade (Section 4.0).

Previous team repositories, which we analysed in Section 7.1: **VHR Motors**: github.com/RAJAS2025/VHR-WRO-FE-June-2025 · **The Robertos**: github.com/Abdu1Hak/WRO-2025 · **This project**: github.com/JumpyTruck/WRO-FE-2026-Sin-Theta



Figure 1.1 — Team SinΘ (Rajas, Harshad, Aarav)

1.2 The Challenge

Open Challenge: the robot drives three laps of the track on its own. The inner walls can be placed at different distances, so it must hold a safe line through straights and corners without hitting a wall. The driving direction (clockwise or counter-clockwise) is random. **Obstacle Challenge:** the same track with red and green traffic signs (pillars) placed at random. The robot must keep red pillars on its right and green pillars on its left. It starts inside the magenta (pink) parking lot, completes three laps and then parallel-parks back into that lot. The goal is a fast, accurate, reliable vehicle that combines mechanical engineering, electronics, computer vision, programming and control.

Teams are also judged on their Engineering Journal and GitHub repository. The 2026 rubric scores five areas (up to 6 points each, 30 in total). Judges look at the quality of reasoning, testing, iteration and reproducibility rather than at length. The table below maps each area to where it is covered in this journal:

Rubric Area	What It Covers	Section
Mobility & Mechanical Design	Chassis, drive and steering systems, mechanical decisions, manufacturing	9.0
Power & Sensor Architecture	Power system, sensors, wiring, placement, calibration	10.0
Software Architecture & Obstacle Strategy	Code structure, algorithms, navigation, obstacle handling, parking	11.0
Systems Thinking & Engineering Decisions	Testing, trade-offs, FMEA, failures, iterations (V1 → V2 → V3)	12.0
Reproducibility & GitHub Quality	Documentation that lets others understand and rebuild the robot	13.0

2.0 Meet The Team



Serious Picture



Funny Picture

Member	About
Aarav Daudia Grade 12, St. Joan of Arc S.S.	Passionate about projects that blend engineering, technology and design, whether modelling mechanical parts in CAD, programming robots, wiring electronics, or building websites and machine-learning tools. Drawn to projects that turn curiosity into something real and useful. Second year in WRO FE; previous experience in WRO FE, VEX, WRO RoboSports, FLL, FRC and Zone_01. Website: crimeinsight.pages.dev
Harshad Mahajan Grade 12, John Fraser S.S.	Second year competing in WRO. Interested in coding, building robots and exploring creative ideas; also enjoys sports and travelling. Previous robotics experience: WRO FE 2025 and Zone_01 2024. LinkedIn: linkedin.com/in/harshad-mahajan-9580362ba
Rajas Salunke Grade 12, Fletchers Meadow S.S.	Third year competing in WRO: RoboMission (Junior category) and one year of Future Engineers, making this his second year in FE. LinkedIn: linkedin.com/in/rajas-salunke-632794422

3.0 Our Journal & Competitive Goal

This journal documents the complete development of our robot, from first ideas through testing, failures and improvements to the final International system. It is organised around the five rubric areas above. Rather than only showing what we built, we explain **why** we made each decision, what testing showed, and how the design changed. The journal and our GitHub repository together form a complete record of the project: the GitHub holds the files (CAD, STL, schematics, code) and the short version of each decision, and this journal holds the full reasoning, data and history.

Competitive goal: design and build a fully functional autonomous vehicle completely from scratch: a fast, reliable, consistent robot that completes both challenges. Beyond results, we use the season to learn the full engineering process (mechanical, electronics, programming, vision, control, testing) and to improve through failure and iteration.

HOW THE JOURNAL MAPS TO THE REPOSITORY

Repository folder	Contents	Journal section
`code/`	Nationals code (`nationals/`), International code (`internationals/`), Arduino firmware, LAB adjuster, serial test code, random layout generator, setup guides (`md/`)	11.0, 13.3
`electrical/`	Final schematic and component-by-component electrical design notes	10.0, 13.2

Repository folder	Contents	Journal section
`models/`	STL and PNG files for every printed part (Nationals set and `Inters Robot Files/`), print settings and materials	9.0, 13.1
`v-photos/` · `t-photos/`	Robot photos (six views of both robots, CAD renders) · team photos	Abstract, 4.0, 12.4
`README.md`	Summary of every rubric area, parts list, rebuild guide	All

4.0 From Nationals to Internationals

Our season had two competition robots. The **V2 National robot** proved the core architecture: camera-only vision, Pi + Nano split, Ackermann steering, metal differential and the PCB V3 power system. It also showed us exactly where we were losing time and consistency. The **V3 International robot** keeps every part of that architecture that worked and re-engineers the parts that did not. In short: we moved the camera lower and further back, rebuilt the rear drivetrain as one casing, enclosed the Raspberry Pi, widened the steering range and rewrote the software as one modular program with variable speed.



Figure 4.1 — Robot evolution, left side: V1 prototype → V2 Nationals → V3 Internationals

Area	V2 — Nationals	V3 — Internationals	Effect
Camera height	Lens on tall stand (≈267.5 mm)	Lens at **230 mm** (−37.5 mm)	~15% less view of objects beyond the field
Camera position	Further forward	**18.3 mm rearward**, 58.8% of length from nose	Pillars stay in view ~340 ms longer
Rear drivetrain	Separate gear holder + two axle supports	One **rear axle casing** holding axles, differential and motor adapter	Less friction, more stable drive
Pi mounting	Open holder	**Enclosed Pi case**, mounted lengthwise, serial port facing rear	Protection; no cables sticking out
Steering range	Centre 97, ±43 servo units	Centre 105, **±60** servo units, revised linkage parts	Tighter corners and parking
Speed	Fixed (PWM 200)	**Pillar-aware variable speed** (255 far / −4 per frame near / 200 cruise)	Obstacle run ~70 s → **~50 s** at ~95%
Software	Two scripts (Open, Obstacle), 1,692 lines	One modular package; **auto-detects challenge**; profiles; video logging	Faster tuning, fewer errors

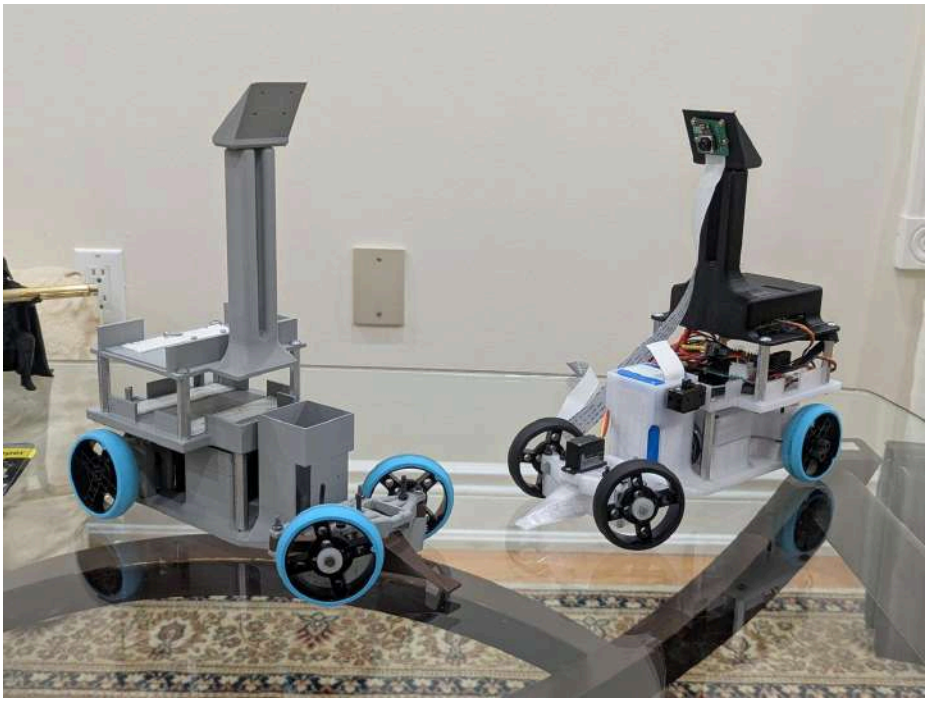


Figure 4.2 — Our two robots this season: the original Nationals design (left) and the evolved International design (right)

The full engineering reasoning, the test evidence and the problem → cause → fix table for each change are in **Section 12.4 (Iterations, V2)**. Performance videos of the International robot are linked in Section 11.11.

5.0 Finance

We set a **$500 CAD** budget at the start of the season. Most of it went to compute (the Raspberry Pi 4B alone is 48% of the core parts cost). We kept the robot cheap by designing and printing our own structural parts (only $5.11 of filament for the final robot), using one camera instead of several sensors, and reusing a vacuum-robot replacement battery pack. Prices below match the parts list in our GitHub README.

5.1 Bill of Materials (BOM)

#	Component	Category	Qty	Supplier	Line Total
1	Raspberry Pi 4B	Compute	1	pishop.ca	$230.95
2	Arduino Nano	Compute	1	pishop.ca	$19.95
3	Raspberry Pi Camera Module 3 Wide	Sensing	1	pishop.ca	$50.95
4	12V 37D Metal Gearmotor (with encoder)	Actuation	1	robotshop.com	$26.88
5	Adafruit DRV8871 Motor Driver	Actuation	1	adafruit.com	$7.50
6	8.4V High-Torque Coreless Servo	Actuation	1	amazon.ca	$17.00
7	8.4V UBEC (2–8S, 8 A)	Power	1	amazon.ca	$25.99
8	5V DC-DC Buck Converter (MINI560)	Power	1	amazon.ca	$2.80
9	IFR 18650 4S1P 12.8V 2000mAh Li-ion Pack	Power	1	amazon.ca	$39.99
10	Rear Metal Axles (BJ-316L, metal 3D print)	Actuation	2	metal print service	$16.50
11	3D Printed Parts (final robot)	Actuation	—	printed in-house	$5.11
	Core parts total				**$443.62**

5.2 Miscellaneous Mechanical Parts

Component	Purpose	Line Total
Filament across all iterations (PLA, PETG, ~1.25 rolls)	Prototypes, V1, V2 and V3 parts	$15.00
VEX Standoffs (1", 1½")	Spacing between the three plates	$1.50
Bearings (×4)	Rear axle support, low friction	$1.00
Screws, Fasteners (M2, M3)	All plate and part mounting	$1.50
VEX Nylon Spacers (¼", ⅛")	Wheel spacing, steering friction	$0.25
LEGO SPIKE Prime Wheels (×4)	All four wheels	$8.00
Metal Differential	Rear drivetrain	$8.00
Total		**$35.25**

5.3 Budget vs. Actual

Category	Budgeted	Actual	Variance
Compute	$300.00	$250.90	+$49.10
Sensing	$50.00	$50.95	−$0.95
Actuation	$75.00	$72.99	+$2.01
Power	$75.00	$68.78	+$6.22
Core parts subtotal	**$500.00**	**$443.62**	**+$56.38**
Mechanical hardware + iteration filament (paid from the remaining $56.38)	—	$35.25	—
Project total (matches GitHub parts list)	**$500.00**	**$478.87**	**+$21.13 under budget**

Only one category went over budget, and only by $0.95 (Sensing), because we chose the Wide version of Camera Module 3 for its larger field of view.

6.0 Season Workflow

Scheduling & coach meetings: at least two team members worked on the robot every day, so development continued even when the full team was not available. Regular coach meetings reviewed progress and challenges. **Deadlines:** personal and team deadlines for each major milestone prevented last-minute work and divided the load. **Self-evaluation:** we regularly compared the robot and documentation against competition requirements, following a **plan** → **build** → **test** → **evaluate** → **improve** cycle that left time to properly test and refine the robot.

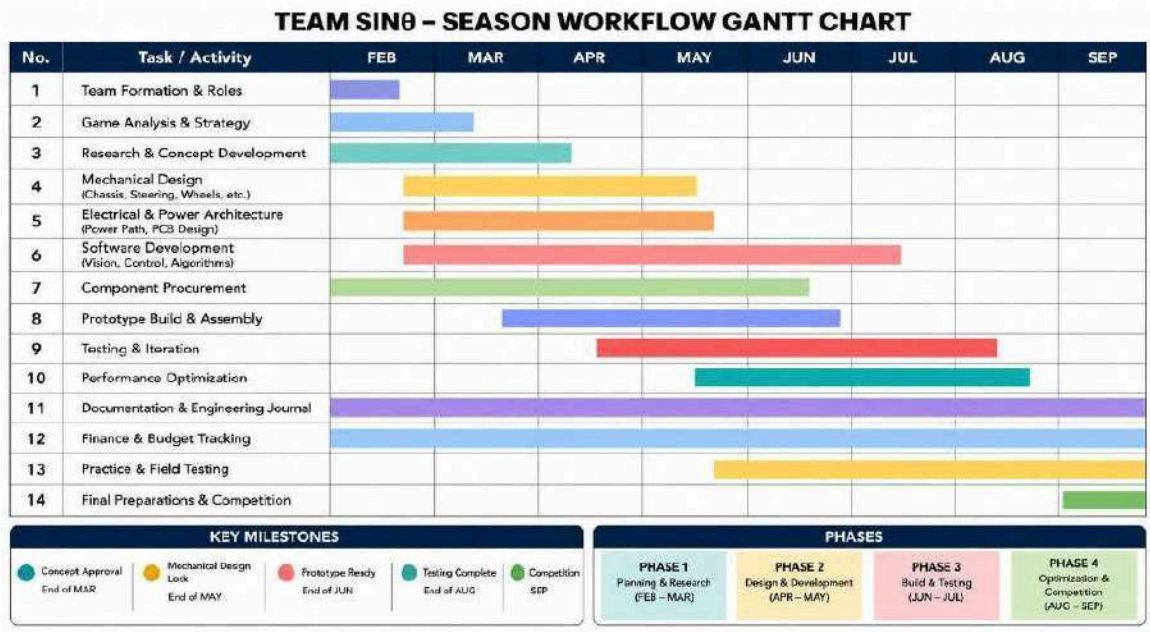


Figure 6.1 — Season workflow Gantt chart (phases 1–5, February → National competition)

The Gantt chart covers the season up to Nationals. After Nationals we added a sixth phase, **Nationals → Internationals**, run in the same plan → build → test cycle. The table summarises what each phase was for. For internationals, we also used GitHub Projects and Issues to be more organized and debug issues better (see Section 13.6).

Phase	Main work	Outcome
1. Planning & research	Team roles, game analysis, previous-robot analysis, concept sketches	Concept chosen, BOM drafted, budget set
2. Design & development	CAD of chassis, steering and drivetrain; electrical architecture; first software	V1 prototype built and driving
3. Build & testing	PCB V1 → V3, Ackermann redesign, vision pipeline (HSV → LAB)	Full 3-lap run with parking
4. Optimisation	Tuning gains, camera angle, consistency tests, documentation	≥90% success over repeated runs
5. Competition prep (Nationals)	Practice on random layouts, checklists, spares	V2 robot competed at Nationals
6. Nationals → Internationals	Camera relocation, rear axle casing, Pi case, steering range, modular code, variable speed	Obstacle run ~50 s at ~95% accuracy

7.0 Game Analysis

7.1 Previous Competition Analysis

We analysed both of our teams' previous WRO FE robots to decide what to keep, what to fix and which lessons to carry into 2026. This analysis directly produced most of our first design requirements (see the "Lesson → 2026 decision" column).

	Team 1 — VHR Motors (Harshad & Rajas)	Team 2 — The Robertos (Aarav)
Strengths	Sharp steering; gearing (fast even with low-RPM motor); well-organised, commented code	Modular chassis (easy to assemble/repair); Raspberry Pi 4B with OpenCV; accurate, strong metal differential

	Team 1 — VHR Motors (Harshad & Rajas)	Team 2 — The Robertos (Aarav)
Weaknesses	HuskyLens camera (poor colour detection, no custom processing); bad centre of gravity (tipping); wheel skid in turns (parallel steering, no Ackermann); ToF sensors slowed decisions; monocoque chassis hard to repair; weak, inaccurate 3D-printed differential	Low motor torque and RPM; multiple fragile components (double-arm steering, plastic axles)
Lessons	Steering, gearing and clean code matter, but so do stability, grip, serviceability and drivetrain design	Modular chassis + Pi/OpenCV + metal differential are a strong base, but software must be efficient, the drivetrain faster, and parts stronger

Lesson	2026 decision	Where
HuskyLens could not do custom processing	Pi Camera 3 Wide + our own OpenCV LAB pipeline	10.6, 11.2
Tipping from a high, uneven CG	Battery low on the centreline, motor central, equal wheels	9.2, 9.3
Wheel scrub with parallel steering	100% Ackermann geometry (12° / ~9°)	9.7
ToF sensors slowed decisions	Camera-only, ~150 ms loop	10.6, 12.1
Monocoque hard to repair	Modular three-plate stack	9.2.2
Printed differential / axles wore and snapped	Metal differential + stainless-steel axles	9.4–9.6
Low torque/RPM drivetrain	510 RPM 37D gearmotor, direct drive	9.4, 10.1

7.2 Strategy & Approach

Our core strategy is a robot that **never needs to stop** during the laps. It navigates continuously and decides in real time, keeping speed high without giving up **accuracy** (correct decisions), **consistency** (repeatable across runs and layouts) or **speed** (fast while stable). We also studied the documentation of strong international teams:

Team Studied	What Inspired Us
Double XX	Simple, effective, accurate strategy and software; straightforward, reproducibility-focused GitHub
Storms NGR	Level of detail and accuracy in GitHub documentation; easy-to-follow engineering decisions
Greenbotics	Servo clamping for efficiency and consistency, which pushed us toward smoother, more consistent solutions (our steering clamps and post-pillar clip)
KMIDS GFM	Depth of GitHub docs with photos and GIFs, which influenced how we present our work visually

7.3 Our Final Approach

Combining these lessons, our approach centres on **continuous movement**: not simply the fastest robot, but one that drives quickly, decides accurately and repeats its performance consistently. To make sure "consistent" meant consistent on *any* layout and not just the ones we happened to practise on, we wrote a **Random Combination Generator** (`code/Random Combination Generator/`). It follows the official WRO 2026 Obstacle Challenge randomisation procedure:

1. **Driving direction:** clockwise or counter-clockwise, chosen at random.
2. **Single-sign section:** two coin tosses pick North, West, East or South.
3. **Single-sign colour:** a third coin toss picks green (heads) or red (tails).
4. **Traffic-sign cards:** of the 36 rulebook cards, cards 9 and 10 are the single-sign cards; the unused one is removed, the remaining 35 are shuffled and three are drawn without replacement for the other sections.
5. **The parking lot section and starting position** are then placed, and the full layout is drawn on top of the field image.

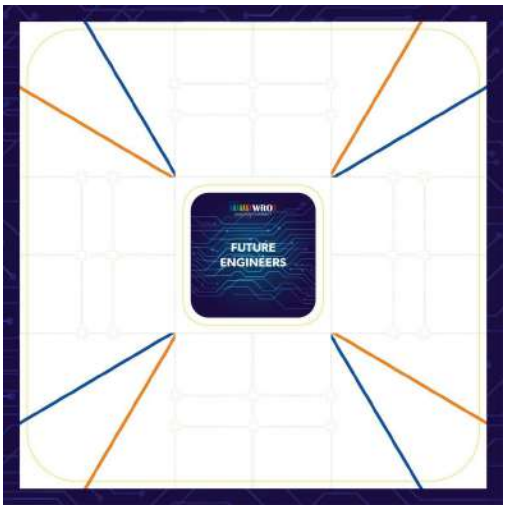


Figure 7.1 — Field image used by the Random Combination Generator

We run the generator to test the robot across obstacle combinations and tweak the code for overall consistency, instead of tuning only on hand-picked layouts. Closely spaced pillars were the hardest case, because the robot can lose sight of one pillar before its body has fully passed it. This is one of the reasons the camera was moved rearward for V3 (Section 12.4).

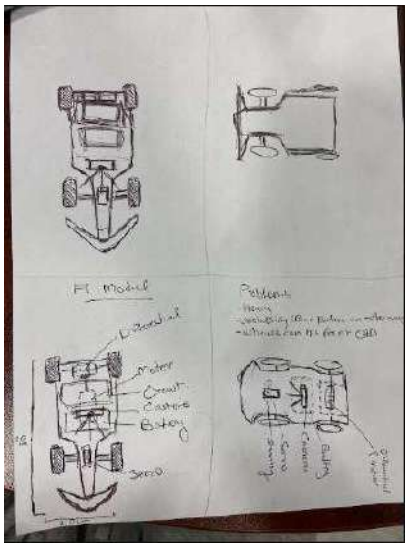
8.0 Research & Concept Development

8.1 Mechanical: we researched chassis layouts, steering geometries, wheel configurations, drivetrains and differentials. Formula 1 cars inspired a large length-to-width ratio for predictable cornering and room to fit pillars between the robot and the wall. Priorities, in order: stability, steering accuracy, speed and serviceability.

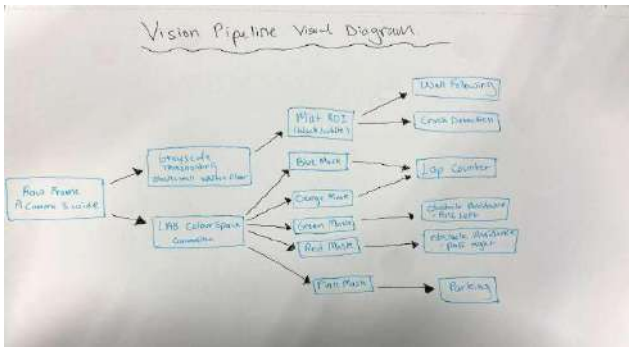
8.2 Power & Sensors: we studied batteries, motors, motor drivers, voltage regulation and power distribution, and compared distance sensors (ToF, ultrasonic) against vision. The aim was a system that reliably supports every subsystem without unnecessary complexity. This research is why our final design has three separate power rails (12.8 V motor, 5 V logic, 8.4 V servo), although we only learned how important that separation is through failure (Section 12.3).

8.3 Software: we explored computer vision (colour spaces, contours, region-of-interest methods), sensor processing, navigation and control approaches (P, PD, PID). We designed real-time decisions that keep the robot moving continuously, and around a state machine so each part of the run could be tested on its own.

8.4 Initial Concepts: before final CAD we sketched the overall layout to place the major components and catch design problems before manufacturing. The sketches already show the key decisions that survived to V3: rear-wheel drive with a central motor, front Ackermann steering, a stacked plate structure and a raised central camera.



Initial concept sketches



Early vision pipeline sketch (refined in 11.2)

9.0 Mobility & Mechanical Design

This section covers how the robot moves: chassis structure, drivetrain, steering geometry, wheels and differential, and how every part is manufactured. Each part was chosen and iterated to maximise turning precision, stability and reliability while keeping the robot compact enough for the WRO Future Engineers mat. Unless noted, descriptions are of the **V3 International robot**; where a part changed from V2 Nationals we say what changed and why.

9.1 Mechanical Requirements

Competition Constraints (WRO FE)	Our Own Design Targets
Max size 300 × 200 mm · Max height 300 mm · Max weight 1.5 kg · Steering on the front axle, no differential (tank) drive · One drive motor	Compact footprint · balanced, low centre of gravity · tight tolerances · easy to assemble, repair and modify · every layer removable without touching the others

Design priority order: Compactness ⟶ Stability ⟶ Precision ⟶ Serviceability

9.2 Chassis

9.2.1 Dimensions — F1-Inspired Layout

Components came first. The motor, differential, camera, PCB and Raspberry Pi were chosen by their real dimensions, and the chassis was sized around them with roughly a 50 mm margin on each side, using the F1 ratio (long wheelbase, narrow track) as the target shape. Placement was fixed before final sizing: the **motor mount** low, central and in line with the differential (no idler gears, like a mid/rear-engined F1 car), and the **battery** low on the centreline ahead of the motor so the two heaviest parts balance each other. **Final footprint: 263 mm (L) × 105 mm (W)**, well inside the 300 × 200 mm limit, with margin for later trims.

Dimension	Value	Effect
Overall width	105 mm	Narrow enough to fit between pillars and walls; centres mass on the long axis; reduces roll and weight transfer in corners
Overall length	263 mm (6.09" nose ⟶ camera + 4.26" camera ⟶ rear)	Better straight-line stability at speed
Wheelbase	7.18" (182.4 mm)	Used in the Ackermann calculation (9.7); a long wheelbase needs a smaller steering angle for the same turning radius

Dimension	Value	Effect
Front track width	3.00" (76.2 mm)	Sets the inner/outer wheel angle difference for 100% Ackermann

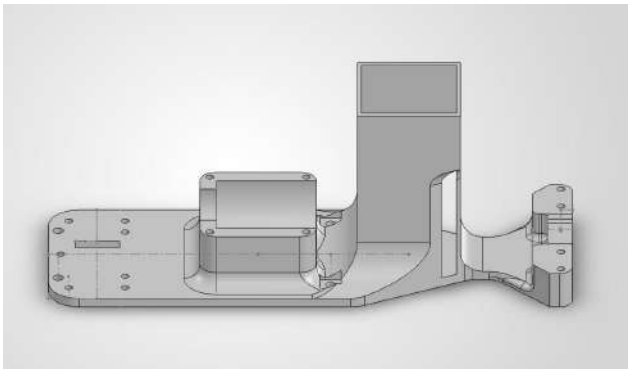


Figure 9.1 — V3 base plate (chassis) CAD

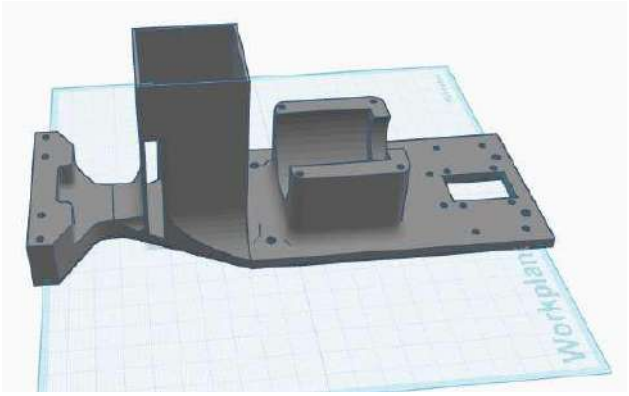


Figure 9.2 — V2 base plate CAD, for comparison

9.2.2 Chassis Architecture — Monocoque vs Modular

Option	Advantages	Disadvantages
Monocoque	Rigid, fewer fasteners, slightly lighter	Repairing one part often means taking apart or reprinting the whole structure
Modular ✓	Sections swapped or redesigned on their own; faster iteration	More joints and fasteners, small weight penalty

Evidence: with VHR Motors' monocoque, a single drivetrain fault meant partly taking apart unrelated sections, which cost repair time during test cycles. The Robertos' modular robot was easy to update and swap parts on, so modular won. Modularity paid off again between Nationals and Internationals: we replaced the whole rear drivetrain (9.5), the Pi holder (9.8) and the camera tower without redesigning the plates that already worked. In a season built on frequent design–test cycles, that matters far more than the small weight saving of a monocoque.

9.2.3 Additional Features

1. **Raised nose:** levels the front and rear axles, with a cut-away giving the servo output room to steer.
2. **Battery holder:** low and on the centreline, ahead of the rear motor, so it counterbalances the motor and keeps the combined centre of gravity near the middle of the chassis. Front-corner cut-aways reduce tyre rub at full lock.
3. **Central motor:** the heaviest moving part, with everything built around it. Together with rear-wheel drive this mirrors a stability-focused mid/rear-engined race car. (The Robertos' vertical motor mount was not possible with this motor's size.) The holder has six mounts and is split into upper and lower halves that clamp the motor, secured by four screws. V3 adds a **motor cover top** printed part.
4. **Rear mounting points:** cut-outs and mounts for the rear axle casing, with a large central cut-away so the differential spins freely.

9.3 Wheels

Both previous teams ran small front and large rear wheels. The Robertos used this for positive rake to help the camera view; this year we increased the camera tilt instead. Both teams reported **oversteer under hard cornering**, caused by the size difference shifting rolling radius and CG height from front to back.

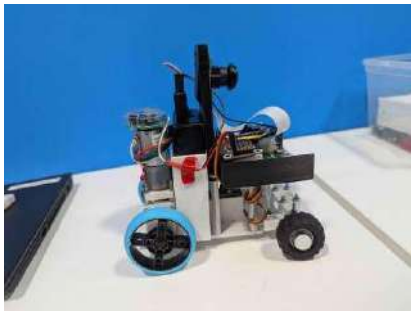
Setup	Rear	Front
The Robertos	57 mm	35 mm

Setup	Rear	Front
VHR Motors	40 mm	15 mm
Sin Theta	**56 mm**	**56 mm**

We chose the **LEGO SPIKE Prime wheel (56 mm × 14 mm) on all four corners**. It gave the best balance of grip, clearance and compactness: bigger wheels hurt turning, smaller ones lose ground clearance. Equal sizes keep CG height level and let one turning-radius table apply to both axles in our Ackermann calculation. **Trade-off:** we gave up VHR's higher top speed per motor rotation in exchange for consistent handling.



VHR Motors (mixed sizes)



The Robertos (mixed sizes)



SPIKE Prime 56 mm, all four corners

9.4 Differential & Drivetrain

Option	Advantages	Disadvantages
3D-Printed	Easy to customise and make	Gears wear fast and mesh poorly; lower strength and accuracy
LEGO	Easy to assemble and modify	Bulkier, sits higher (raises CG), limited strength for our drivetrain
Metal ✓	Strong, precise, meshes smoothly, durable, sits lower	Heavier and less customisable

Our motor connects **directly** to the differential, with no gearing. VHR used a 1.5:1 stage (36T spur into a 24T input) on a 175 RPM motor to step up RPM; The Robertos ran direct drive from a vertical motor. We kept direct drive with a 510 RPM motor:

Setup	Motor	Output	Wheel speed (56 mm wheels)	Torque at differential
The Robertos — direct	236 RPM	236 RPM	$\pi \times 0.056 \times 236 \approx 42$ m/min (0.69 m/s)	Full motor torque (5.2 kg·cm)
VHR — geared	175 RPM + 1.5:1	≈263 RPM	$\pi \times 0.056 \times 263 \approx 46$ m/min (0.77 m/s)	Reduced by gear ratio
Sin Theta — direct	510 RPM	510 RPM	**0.598 m/s** max (calculated, see below)	Full torque, no reduction

The previous teams' speeds are simple no-load estimates. For our robot we did the full drivetrain calculations (working shown on GitHub, "Drive System" and "Motor & Drive System"):

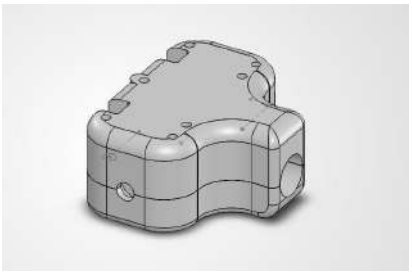
Calculation	Result	What it tells us
Required motor RPM for a 28 s Open Challenge	**452 RPM**	Our 510 RPM motor clears the requirement with margin

Calculation	Result	What it tells us
Required motor torque to reach top speed in 0.353 s	**0.190 kg·cm**	Far below what the motor provides, so we never run near stall
Maximum robot speed (under load)	**0.598 m/s**	Gives a **21.7 s** theoretical best Open Challenge time
Maximum acceleration	**15.36 m/s²**	Top speed is reached in 58.4 ms, so speed changes are practically instant

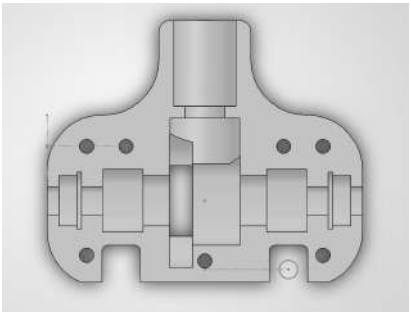
Torque trade-off: a faster motor usually gives up stall torque, so we rely on the differential to deliver torque with zero gear-stage loss rather than regaining it through reduction. **Test result:** VHR's 1.5:1 stage into a 3D-printed differential showed measurable gear and housing wear over the season, with mounting-hole play increasing under cornering loads. The Robertos' metal differential ran a full season with no comparable wear, and ours has shown **minimal wear after 100+ total laps**. Removing the gear stage removes VHR's wear point entirely, the metal housing adds durability, and we still beat VHR's geared wheel speed.

9.5 Rear Axle Casing (V3) · 9.6 Rear Axles

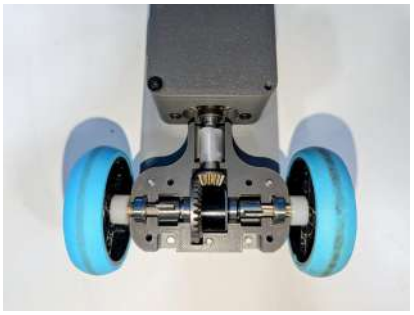
On V2 the drivetrain was held by separate parts: a gear holder for the differential and two axle supports with bearings. They worked, but every part had to be aligned with every other part by hand, and the motor adapter binding problem (12.3.3) showed how sensitive the gear mesh was to that alignment. For V3 we designed a single **rear axle casing**: a two-part housing that holds the rear axles, the metal differential and the motor adapter in one rigid body. Because the bearing seats, differential pocket and adapter bore are all modelled in the same part, they are aligned by the CAD instead of by assembly. This lets everything mesh together, reduces friction in the axles, differential and adapter, and makes the drive system more stable.



Rear axle casing — exterior



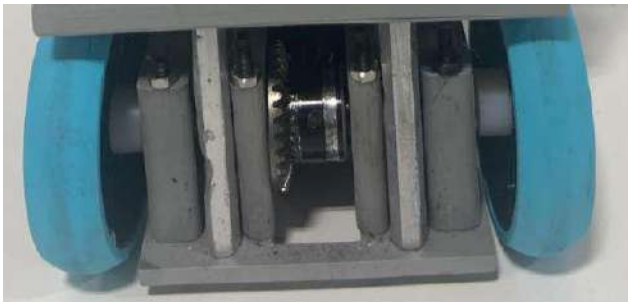
Inside: bearing seats, differential pocket, adapter bore



Full rear axle assembly — installed

Axle material	Flex under load	Alignment shift
3D-printed	0.8 mm	~2.5° during acceleration; snapped after repeated runs
Metal ✓	0.2 mm	Within tolerance; minimal twisting

The Robertos' similar axles were fragile because they were printed in plastic, so we had ours custom printed in **stainless steel (BJ-316L)** by a metal 3D-printing service: a 4× reduction in flex. The custom axles adapt the differential output to the LEGO wheels. **VEX nylon spacers** on the rear axles push the wheels outward so they do not rub the chassis, and LEGO rubber fittings with a black peg secure each wheel.



Metal differential



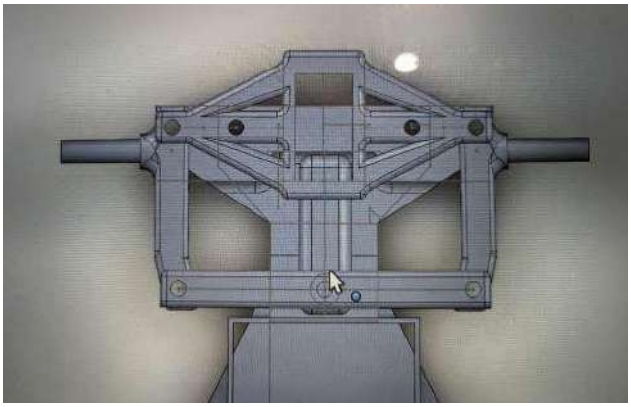
Stainless-steel (BJ-316L) rear axles

9.7 Steering System (Ackermann)

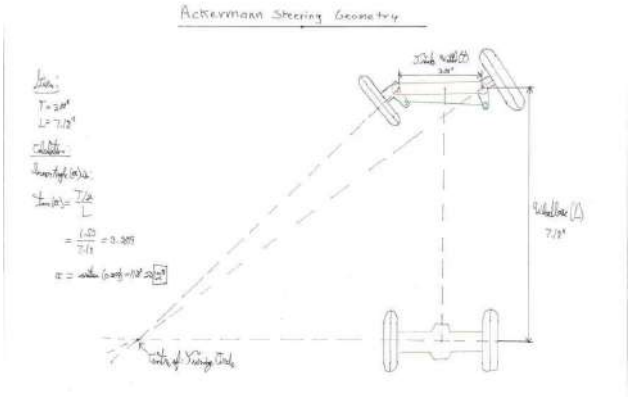
The Robertos' double-arm steering allowed a high-rear servo and a large maximum angle, but it was extremely fragile and bulky. VHR's **parallel steering** (both wheels at the same angle) made the inner wheel scrub against its natural arc on sharp corners, causing skidding and loss of grip. We chose **front-wheel 100% Ackermann steering calculated from our chassis dimensions**. In 100% Ackermann the inner wheel turns more than the outer wheel, so every wheel always rolls on its own correct radius and the axes of both front wheels meet at the centre of the rear axle. This removes scrub, at the cost of a more complex linkage to make and align.

THE CALCULATION

Ackermann geometry is set by two chassis dimensions: the **wheelbase (7.18")** and the **front track width (3.00")**. For every wheel to roll on its own radius, the steering arms must be angled so that lines drawn through them meet at the centre of the rear axle. Our hand calculation (below, with the full working on GitHub) gives a **target steering angle of ~12° at full lock** for true 100% Ackermann. The knuckle arms are printed at that angle, so one servo produces the larger inner-wheel angle and the smaller outer-wheel angle automatically.

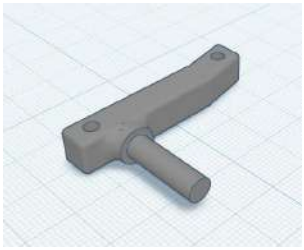


Ackermann linkage CAD, front view

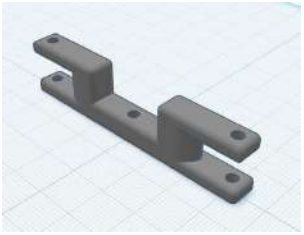


Ackermann geometry hand calculation

Steering knuckle: the 12° arm angle produces **100% Ackermann**: lines drawn through each front wheel meet exactly at the centre of the rear axle (95% Ackermann would meet 5% of the wheelbase further forward). **Cross link:** joins the knuckles from above and below for a secure fit, turning servo rotation into steering. **Steering T:** extends the servo output to reach the cross link (placed further out to maximise steering angle) with a flush, low-friction attachment. **Steering holder (case):** holds the knuckles beneath it and mounts the servo **inverted** so its output and the knuckles share one plane, which reduces complexity and increases compactness; a six-triangle pattern adds strength while cutting weight. Nylon spacers at the pivots cut friction.



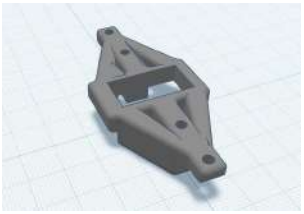
Steering knuckle



Steering cross link



Steering T



Steering holder

V3 changes: the International build uses a shortened steering T, a cross link and servo holder with added clearance gaps, and a revised Ackermann steering holder (files `Steering T - Short`, `Steering Cross Link - Gap`, `Servo Holder Gap Final`, `Ackerman Steering Holder` in `models/Inters Robot Files`). Together with a re-calibrated servo centre (97 → 105) this widened the usable steering range in software from ±43 to **±60 servo units**, giving tighter turns in corners and during parking (Section 12.4).

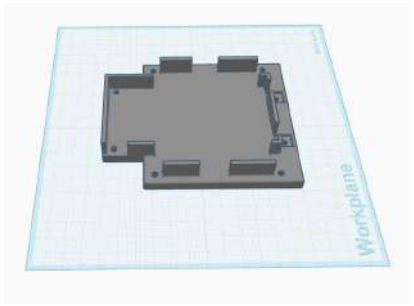
STEERING VALIDATION (FROM GITHUB "MECHANICAL TESTING & VALIDATION")

Test	Method	Result
Ackermann angle verification	Full lock measured with a digital protractor; target 12° inner / ~9° outer	Achieved within **±1°**
Tyre slip at full lock	Compared against our 2025 parallel steering	**<2°** slip vs **12°** with parallel steering
Turning radius consistency	10 consecutive full-lock turns at fixed speed	**11.9 cm ± 0.3 cm**, very consistent
Differential balance	Encoder ticks per lap, left vs right wheel	**156 vs 155** ticks/lap; the differential lets wheel speeds vary correctly in turns

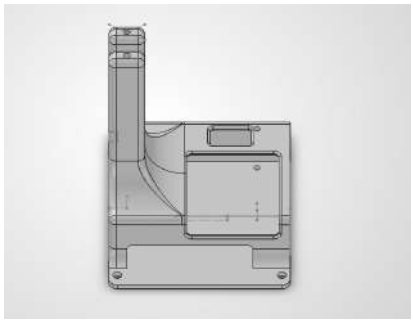
9.8 Electronics Plates

The second plate holds the custom motor-control PCB. The main rectangle seats the PCB and a second, smaller pocket houses the servo regulator (UBEC); the same plate also fits our older PCB as a drop-in backup. The USB serial link to the Pi stays reachable, with clearance to swap parts without removing the plate above.

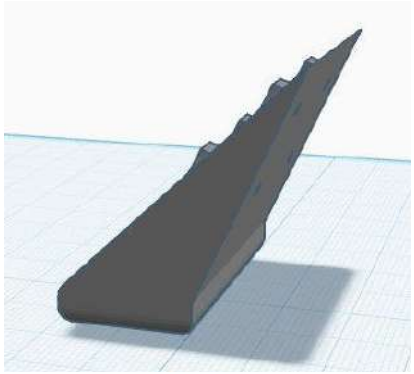
The third plate combines the Raspberry Pi holder and the camera tower in one rigid piece, so vibration or collisions cannot shift the pixel-to-world mapping the vision system depends on. On V2 this was an open holder. For V3 it became an **enclosed Pi case** ("Pi Reverse") that protects the board. It keeps cut-aways for the wiring and is **mounted lengthwise with the serial port facing rearward**, so cables no longer stick out where they could catch a wall or pillar. The **camera mount** is a separate part on top of the tower. Angle selection needs many iterations, and reprinting the whole Pi holder each time would waste time and filament. It sits at a **155° tilt** with flat rear faces for flush M3 mounting.



Second plate (PCB + UBEC)



V3 third plate: enclosed Pi case + tower



Camera mount (155°)

9.9 Robot Height & Camera Placement

Camera placement decides what the robot can see, so we treated it as a mechanical design problem with measurable outcomes. It was the single biggest mechanical change between Nationals and Internationals.

Parameter	V3 value	Reasoning / validation
Lens height	**230 mm** (lowered 37.5 mm from V2)	On V2 the high lens saw colours beyond the field borders, which added noise to the masks. Lowering it gives **~15% less view of beyond-field objects** while still seeing both walls.
Camera tilt	**155°** down-angle	Clear view of both near lane markers and distant obstacles; field of view ~160° horizontal, covering the full track width from **10 cm to 200 cm** ahead.
Lateral position	Centred on the robot centreline	No left/right offset compensation needed in image processing; lane-marker detection is symmetric with no bias.
Longitudinal position	6.09" from nose, 4.26" from rear (**~58.8%** of length from the nose); moved **18.3 mm rearward** from V2	A more rearward camera keeps a pillar in view for **~340 ms longer**, so most of the robot has passed it before the robot starts looking for the next one, reducing the chance of hitting the adjacent pillar.

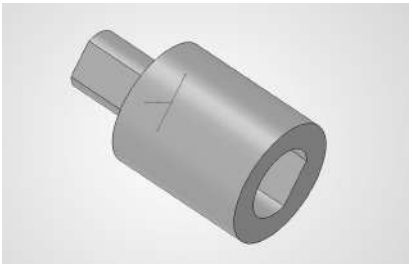
Tilt	Near obstacle	Far obstacle	Overall field visibility	Chosen
135°	Excellent	None	Poor; can only see near	No
145°	Good	Limited	Okay; sees far obstacles very late	No
155°	Good	Very good	Best coverage	**Yes**
165°	Limited	Very good	Good, but loses obstacles too soon → crashes	No
175°	Poor	Excellent	Poor; cannot see in front of itself, always crashing	No

Front vs middle vs rear mount: VHR's rear mount added lag between what the camera sees and where the front wheels are. A front mount removes that lag but sits too close to the walls in tight corners. We started from The Robertos' **middle placement**, which keeps the camera roughly equidistant from both walls through a turn, then moved it slightly rearward for V3 based on the pillar-visibility result above.

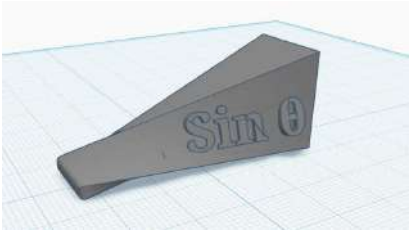
9.10 Motor Adapter · 9.11 Bumper

Motor adapter: the motor has a D-shaft, but the differential has a square input, so an adapter is required. Slots firmly secure the input and maximise torque transfer. It is printed in **PETG at 100% infill**: it couples the motor shaft directly to the differential and sees constant torsional shear and impact stress, so it needs maximum material density and PETG's toughness to avoid stripping or grinding. In V3 it sits inside the rear axle casing, which holds it in line with the differential input.

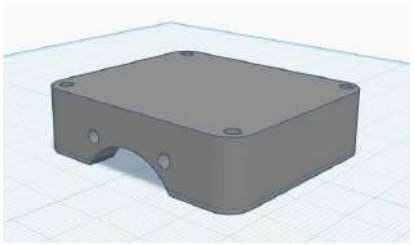
Bumper: a protruding nose bumper (1) stops the robot getting stuck on a wall or pillar, (2) is fixed with thick tape that acts as a shock absorber, reducing stress on the chassis in crashes, and (3) since an F1-inspired car needs a front wing, it carries our team name **"Sin θ"** so everyone knows which team it is running for.



Motor adapter (PETG, 100% infill)



Front bumper / wing



Motor mount, top half

9.12 Manufacturing & Material Specifications

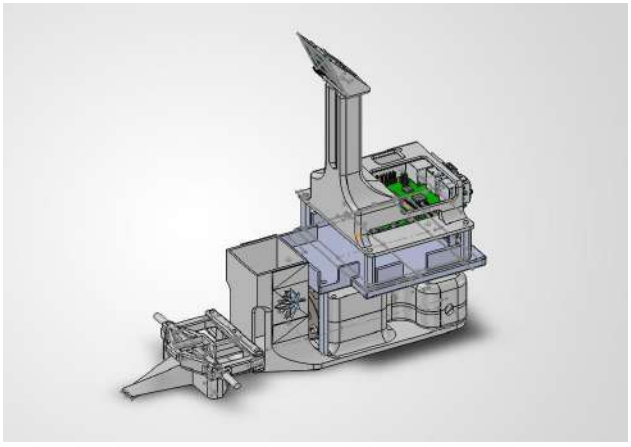
All printed parts follow one standard profile, so a reprinted part behaves exactly like the original. Parts were printed on a **Bambu Lab X1 Carbon**; the full print of all parts takes about 24 hours and uses about $5.11 of filament.

Setting	Value	Why
Infill	**15%** (all parts) · **100%** (motor adapter)	15% balances strength against weight. More infill adds mass, raises the CG and slows acceleration. The adapter carries full motor torque, so it is solid.
Infill pattern	**Gyroid**	Distributes stress equally in all directions (isotropic) and avoids weak planes during wall hits or hard braking
Wall loops	**5**	Much stronger outer shell, bolt holes and snap fits than the default 2 walls; stops motor mounts and steering holders shearing under load
Layer height	**0.2 mm**	Balance of resolution, clean tolerances for the interlocking plates, and layer bonding
Supports	Tree/organic, auto	Only on overhangs (servo mounts, camera brackets, axle casing internals) for clean bridging without hurting accuracy

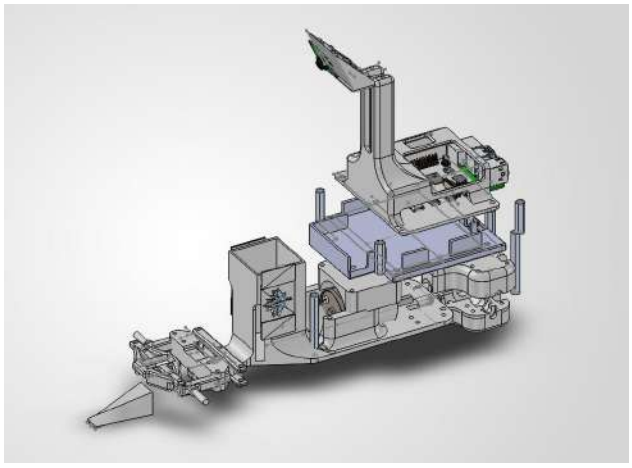
Material	Used for	Why
PLA	Base plate/chassis, second plate, third plate and camera tower, steering system	Strong, dimensionally accurate and light, keeping the robot stable in fast corners
PETG	Motor adapter and critical drivetrain housings	Tougher, better layer adhesion and slightly ductile, so it absorbs shock and torsion from the motor without shattering

9.13 Final Mechanical Assembly & Validation

The final mechanical system combines a modular three-plate chassis, uniform SPIKE Prime wheels, a metal differential and stainless-steel axles in one rear axle casing, calculated 100% Ackermann steering, and an integrated enclosed-Pi and camera tower.



V3 full assembly (CAD)



V3 exploded view (CAD)

Validation	Result
Centre of gravity tilt test	Battery centred and low keeps the CG low and stable; **no tipping in sharp turns even at full speed (0.598 m/s)**

Validation	Result
Durability	**15 consecutive laps** without any printed part cracking; metal differential shows **minimal wear after 100+ total laps**
Steering precision	Linkage keeps its precision throughout extended testing (see 9.7 for angle and radius results)

10.0 Power & Sensor Architecture

The electrical system is built around three ideas: (1) an **Arduino Nano** handles low-level, time-sensitive motor and servo control, turning high-level commands into exact PWM signals in real time; (2) a **Raspberry Pi 4B** handles perception and decisions using a single camera; (3) the two talk over **USB serial**, so perception code and motor code can be developed and updated independently. Power is split into **three isolated rails**, a lesson learned the hard way (12.3).

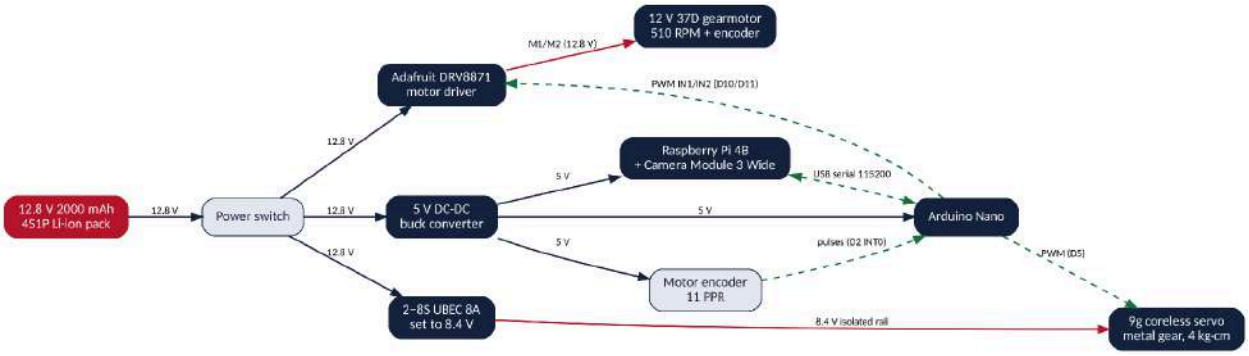


Figure 10.1 — Electrical architecture: power rails (solid) and signals (dashed)

10.1 Motor Requirements / Selection

We set criteria for the drive motor before comparing options: **300–500 RPM** at the motor (with our 56 mm wheels, ~175.93 mm circumference, that is roughly **0.88–1.47 m/s**: fast enough, without making acceleration, braking and precise positioning difficult), a **minimum torque of 1.5–2.0 kg·cm** to move the robot's mass and overcome drivetrain friction with margin, a built-in gearbox and **encoder**, and a compact size.

Criterion	12V 37D Metal Gear Motor ✓	520 DC Gear Motor w/ Hall Encoder
Encoder (required)	Yes, integrated **11 PPR** encoder for closed-loop distance	Yes
Speed (300–500 RPM target)	510 RPM; top of the range, best for lap time	Weaker fit to the target range
Torque (≥1.5–2.0 kg·cm)	Above minimum without running near stall (which causes excess current and heat)	Weaker fit to our torque range
Gearbox (required)	Yes, metal gearbox turns raw speed into usable torque	Yes
Physical size	Compact 37D fits beside battery, PCB, sensors and steering	Weaker match for a compact chassis
Decision	**Chosen: meets every criterion**	Not chosen

The encoder matters as much as the motor. It measures actual wheel rotation in ticks, instead of relying on timed movement, which drifts with battery voltage, friction, load and surface. Every parking move uses encoder ticks for this reason (11.7). At 12.8 V the motor draws **0.40 A normal / 1.00 A peak (5.12 W)**.

10.2 Battery Selection

The motor and DRV8871 are both rated for a 12 V supply, so we targeted ≈12 V: enough to run the motor at full speed with no boost converter, and within the driver's safe range. Four Li-ion cells in series was the minimum to clear 12 V with headroom for voltage sag. We use an **IFR 18650 4S1P 12.8 V 2000 mAh pack**, a replacement pack originally designed for robot vacuums (Lefant/OKP Life). It is compact, light and much safer to handle than a LiPo of similar capacity. With a peak draw of 2.0 A it kept the rail stable through full-load acceleration and steering with **no brownouts**.

Battery	Supplied	Size	Verdict (required ≈12 V)
7.2 V NiMH (6-cell)	7.2 V	Compact, low energy density	No: far short of 12 V; cuts speed and torque; NiMH heavier per Wh
9.6 V NiMH (8-cell)	9.6 V	Larger, heavier for a modest gain	No: 2.4 V short; extra cells too bulky compared with Li-ion
12.8 V 2000 mAh Li-ion (4S1P) ✓	12.8 V	Compact; fits beside PCB, sensors, steering	**Yes:** closest to 12 V; drives motor and DRV8871 directly; light and energy-dense
18.5 V 5S LiPo 2200 mAh	18.5 V	Bulkier, extra series cell	No: 6 V+ over target; risks overdriving 12 V parts (50%+ over-voltage)

Runtime check: at the 1.2 A normal draw from the power budget (10.4.1), 2.0 Ah gives a theoretical ~100 minutes. We measured **75 minutes** of real runtime (which includes peaks from steering and acceleration), well above our 60-minute requirement. The battery carried through unchanged from V1 to V3; every PCB revision was designed around the 12.8 V supply.

10.3 Motor Driver Selection

Driver	Properties	Verdict
Adafruit DRV8871 ✓	6.5–45 V, 3.6 A continuous / 6 A peak, PWM speed + direction, over-current/thermal/under-voltage/short protection	**Yes:** 3.6 A exceeds our ~2 A maximum so it is never the bottleneck; MOSFET design gives nearly full battery voltage to the motor; tiny footprint
Pololu VNH5019	5.5–24 V, 12 A continuous / 30 A peak, current sensing, reverse-voltage and thermal protection	No: 6× our need; unused capability at extra cost
L298N dual H-bridge	5–35 V, 2 A/channel, bipolar transistors	No: ≈2 V internal drop leaves the motor ≈10.8 V; less efficient and hot (it overheated on PCB V1)
Cytron MD13S	6–30 V, 13 A continuous, MOSFET, protection	No: 6×+ our need; unnecessary cost

At 12.8 V the driver itself draws **<0.05 A normal / <0.1 A peak (0.64 W)**. Note that the DRV8871 needs a **5 V PWM** signal to switch reliably; the 3.3 V ESP32 we first used could not drive it, which is one reason the Nano replaced the ESP32 (12.3.2).



12 V 37D gearmotor + encoder



12.8 V 4S1P Li-ion pack

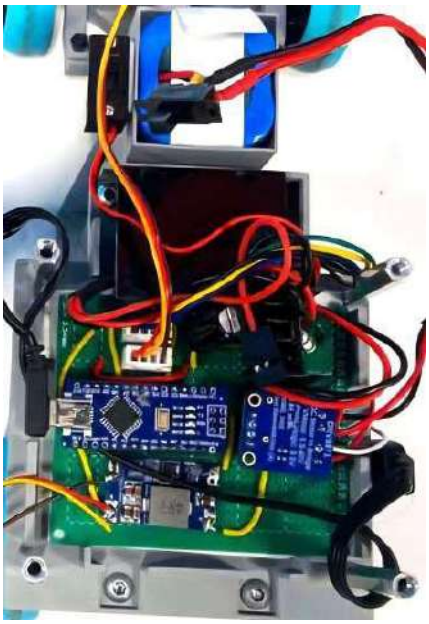


Adafruit DRV8871

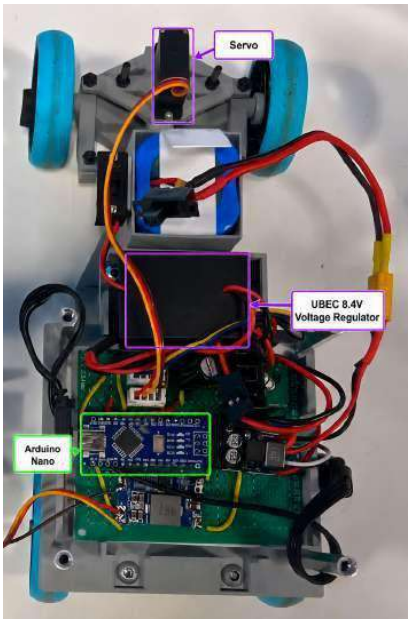
10.4 PCB Version 3: Final Version

Shared logic electronics draw up to **1.5 A peak at 5 V (4.0 W)**. The 7805 linear regulator went into thermal shutdown converting 12.8 V → 5 V at this load (see 12.3.1), so we replaced it with a **5 V switching buck converter** that supplies the same current with far less heat. All 5 V logic (Pi + camera, Nano, encoder) runs from that single converter, which keeps the wiring simple and avoids several separate regulators. The servo separately draws up to **3.0–5.0 A peak**, so it is isolated on a **dedicated 8 A UBEC set to 8.4 V**. After these changes the board stayed within thermal limits under sustained current, and the servo steered under load without stalling. PCB V3 was used unchanged on both the V2 and V3 robots.

Component	Role
Arduino Nano	Slave controller: converts Pi steering/speed commands into PWM for the driver and servo, counts encoder ticks. 5 V buck rail.
Raspberry Pi 4B + Camera 3 Wide	Main processor: runs vision and all decisions, sends commands to the Nano (10.5). 5 V buck rail.
DRV8871	Drives the motor from the 12.8 V rail; logic side fed by Nano PWM, keeping the high-current path separate from logic.
5 V DC-DC buck	12.8 V → 5 V for Nano, Pi, encoder and logic. Replaces both 7805s from earlier PCBs.
2–8S UBEC 8 A	Dedicated 8.4 V rail for the servo only, so servo spikes cannot brown out the board.
12.8 V 4S1P pack	Feeds the DRV8871, buck and UBEC directly through the power switch.
9g coreless servo	Full metal gear, 4 kg·cm, 4.8–8.4 V; powered by the UBEC, commanded by Nano PWM.
Encoder / servo / M1–M2 ports	Encoder port to the Nano (5 V) for closed-loop distance; 3-wire servo port (power, ground, PWM) isolated from the main rail; motor ports carry 12.8 V directly, bypassing both regulators.



PCB Version 3



PCB V3 labelled: Nano, UBEC, servo

10.4.1 PCB Version 3 Power Budget

Component	Input Voltage	Normal Current	Peak Current	Power
12.8 V 2000 mAh 4S1P Li-ion battery	12.8 V	1.2 A	2.0 A	15.36 W
DRV8871 motor driver	12.8 V	< 0.05 A	< 0.1 A	0.64 W

Component	Input Voltage	Normal Current	Peak Current	Power
12 V 37D metal gear motor	12.8 V	0.40 A	1.00 A	5.12 W
5 V DC-DC buck converter	12.8 V → 5 V	0.8 A (out)	1.5 A (out)	4.0 W (out)
Arduino Nano	5 V	0.05 A	0.10 A	0.25 W
Raspberry Pi 4B + Pi Camera 3 Wide	5 V	0.70 A	1.20 A	3.50 W
Motor encoder	5 V	0.02 A	0.03 A	0.10 W
2–8S LiPo UBEC 8 A	12.8 V → 8.4 V	0.33 A (in)	3.5 A (in)	2.8 W (out)
9g coreless servo	8.4 V	0.50 A	3.0–5.0 A	4.2–25.2 W

The budget shows why the servo needed its own rail: its peak (up to 25 W) is larger than everything else combined. On a shared rail, one hard steering correction could pull the 5 V logic down far enough to reset the Nano or the Pi, which is exactly what happened on PCB V2.

10.4.2 PCB Safety — Why No Separate Fuse

- The **buck converter and UBEC** both include built-in over-current and over-voltage protection, so a short or spike downstream is shut down inside the converter before it damages the board.
- The **DRV8871** adds over-current, thermal, under-voltage and short-circuit protection on the motor side, containing a stalled motor or output fault at the driver.
- Since every high-current path already has its own protection, an external fuse would only trip after these had acted, which makes it redundant. Leaving it out keeps the board simpler and lighter, with one less failure point. A **main power switch** between the pack and the board gives a manual cut-off.

10.5 Raspberry Pi 4B + Arduino Nano

The Pi handles all high-level decisions; the Nano handles only low-level, time-sensitive motor and servo control. Using the Camera Module 3 Wide, the Pi identifies lane lines, obstacles and track boundaries in real time, a workload far too heavy for a microcontroller. After each frame it sends steering and speed over serial, and the Nano turns them into PWM. This lets the vision code be updated independently of motor control while holding the cycle at **~150 ms per frame**. The Pi draws **0.70 A normal / 1.20 A peak (3.50 W)** together with the camera.

Why a Nano and not the ESP32 we started with: the ESP32's 3.3 V PWM did not reliably trigger the DRV8871, while the Nano outputs a full **5 V PWM**. Its compact **45 × 18 mm** size and **1–3 second** code upload time also suited our board and fast test cycles. Separating motor control from the Pi also means a slow vision frame can never delay a motor or servo update. The Nano draws only **0.05 A normal / 0.10 A peak (0.25 W)**.

Connection	Pin / setting	Notes
Pi ↔ Nano	USB serial, **115200 baud**	Pi waits 2 s after opening the port, since opening it resets the Nano
Start button	Pi **GPIO 26** (BCM)	Active-low, wired to GND with the internal pull-up
Status LED	Pi **GPIO 16** (BCM)	Lit while the robot waits for the start button
DRV8871 IN1 / IN2	Nano **D10 / D11**	PWM speed and direction
Steering servo	Nano **D5**	Servo library PWM; power from the UBEC
Encoder channel A	Nano **D2** (INT0)	Interrupt on every edge counts ticks; channel B on D7
Camera	Pi CSI connector	Ribbon cable up the camera tower



Raspberry Pi 4B



Pi Camera Module 3 Wide

10.6 Sensor

The final architecture uses a **single sensor**: the Pi Camera Module 3 Wide, chosen for its wide field of view, which gives better line and object coverage. Camera-only, the processing cycle runs at **~150 ms**. When we added ToF and gyro sensors it rose to **~600 ms, a 450 ms (≈4×) increase**. Because the robot has to keep processing and correcting while it drives, that extra latency would make it react more slowly and reduce navigation accuracy. The camera already supplies everything our vision algorithm needs from a single image, so we chose processing speed and responsiveness over redundant distance sensors. Camera placement (height, tilt and position) is covered in 9.9; calibration of the colour thresholds is covered in 11.2 and 13.3.

11.0 Software Architecture & Obstacle Strategy

Every behaviour (wall following, pillar avoidance, crash recovery, lap counting and parking) is a small, independent piece of logic that proposes a steering angle each frame. The **state machine** decides which behaviours may run at each stage of the run, and the **arbitration layer** turns their proposals into one decision roughly every 150 ms. Because the loop repeats that often, a bad decision on one frame is corrected within a fraction of a second on the next.

Stage	What happens	Feeds into
Capture	Camera Module 3 Wide grabs a 640×360 frame at 30 fps (155° tilt)	Vision pipeline (11.2)
Vision pipeline	Frame cropped to 640×280, converted to LAB; colour masks built in parallel; track image and track mask built	Wall following, pillars, crash checks, laps, parking
Behaviour layer	Each behaviour independently proposes a steering angle (or defers)	Arbitration (11.9)
Arbitration	Fixed priority order picks one winning angle; speed is updated	State machine (11.8)
State machine	Decides which behaviours may run, based on the stage of the run	Serial link to Nano
Motor / servo	Nano turns steering + speed into PWM for the DRV8871 and servo	Physical movement

CODE STRUCTURE (INTERNATIONAL VERSION, CODE/INTERNATIONALS/)

For Nationals our code was two long scripts (`OpenChallenge.py`, 459 lines, and `ObstacleChallenge.py`, 1,233 lines) that had to be chosen by hand before each run, and many tuning values were duplicated between them. For Internationals we rebuilt it as **one modular package**. The maths is identical for both challenges; only the numbers change, and they are read from a profile chosen automatically at the start.

File	Responsibility
`main.py`	`RobotController`: the state machine for the whole run (11.8)
`classes/pillar_navigator.py`	Every steering angle: crash checks, pillar avoidance, pink parking wall, three kinds of wall following. Never talks to the motors.
`classes/lap_counter.py`	Counts corners (quarter turns) from the corner-marker masks (11.6)

File	Responsibility
`classes/arduino_comms.py`	All serial commands to the Nano; servo mirroring and steering limits
`classes/video_recorder.py`	Records every run with lap counter, timer and speed drawn on the video (debugging aid)
`utils/constants.py`	Every value shared by both challenges (LAB ranges, ROIs, thresholds, gains)
`utils/challenge_profile.py`	`ChallengeProfile` dataclass: Open vs Obstacle values (speed, gains, ROIs, lap-count timing)
`utils/park_profile.py`	`ParkProfile` dataclass: left-run vs right-run parking values (mirror images with their own tuning)
`utils/image_utils.py` · `utils/enums.py`	Frame processing and masks · `Challenge`, `Direction`, `TurnState`, `WallState`, `RunState`

11.1 Raspberry Pi + Arduino Nano

The robot's brain is split across two boards connected by a USB serial cable: the Pi sends simple text commands and the Nano carries them out, reporting back when a distance move is done. The Pi's CPU is busy with vision. Interleaving time-sensitive PWM on the same processor would risk missed motor timing whenever the scheduler was busy with a frame (full trade-off in 12.1).

Responsibility	Raspberry Pi	Arduino Nano
Camera capture & vision processing	Yes: full OpenCV pipeline	No
Navigation (walls, pillars, parking, laps)	Yes: all high-level logic	No
PWM generation for motor & servo	No	Yes: turns Pi commands into PWM
Closed-loop distance (encoder ticks)	No	Yes: counts ticks, replies when a move is done
Start button & status LED	Yes: GPIO 26 / 16	No

SERIAL PROTOCOL (115200 BAUD, ONE PLAIN-TEXT COMMAND PER LINE)

Direction	Message	Meaning	Example
Pi → Nano	`SERVO,{angle}`	Set steering angle (0–180)	`SERVO,105`
Pi → Nano	`MOTOR,{dir},{speed}`	Drive continuously; dir 0 stop / 1 forward / 2 reverse; speed PWM 0–255	`MOTOR,1,200`
Pi → Nano	`MOVE,{dir},{speed},{ticks}`	Drive a fixed number of encoder ticks, then stop by itself	`MOVE,2,100,1100`
Pi → Nano	`STOP` / `STOP,1`	Coast / brake (hard stop)	`STOP,1`
Pi → Nano	`RESET` · `TICKS?`	Zero the encoder count · ask for the count	`RESET`
Nano → Pi	`DONE,{ticks}`	A MOVE finished; reports ticks actually travelled	`DONE,1102`
Nano → Pi	`DONE,{ticks},TIMEOUT`	MOVE gave up early (e.g. robot stuck)	—

Normal lap driving uses `MOTOR` and `SERVO`, where continuous control matters most. Parking uses `MOVE` and waits for `DONE`, where consistent distance matters more than speed. The Pi only re-sends a servo angle when it changes, which keeps the serial line quiet. The split also lets each codebase be developed and debugged on its own, mirroring the modular chassis (9.2.2).

11.2 Vision Pipeline

Every raw frame is turned into a few simpler images that everything else works from: the **cropped frame**; one black/white **colour mask** each for blue, orange, green, red and pink; and the **track image** (walls black, floor white) with

its **track mask**. Each output has a clear consumer, and colours a challenge does not use are not computed at all (the Open Challenge only builds blue and orange masks; pink is only built while parking).

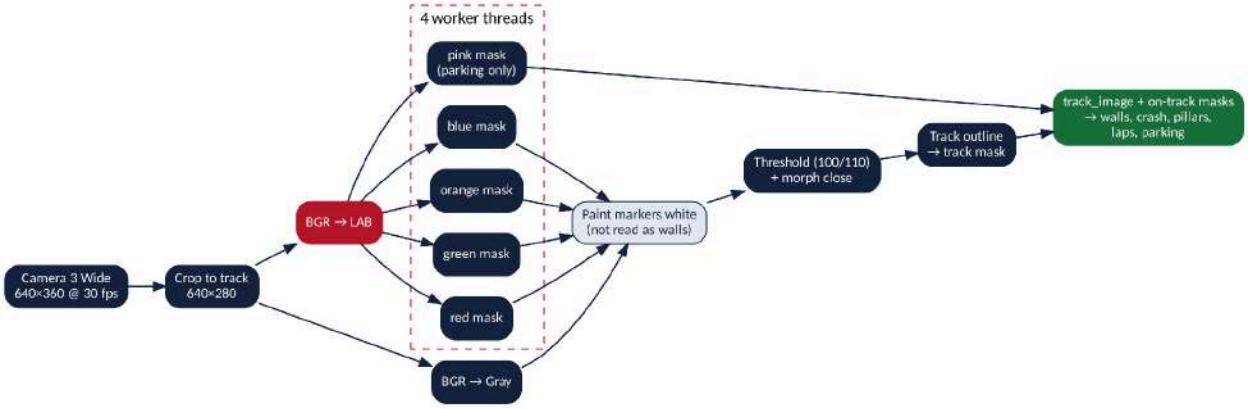
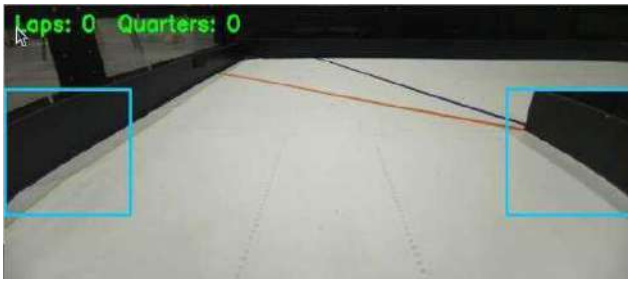
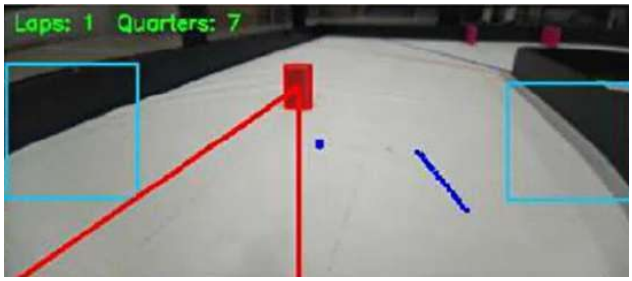


Figure 11.1 — International vision pipeline (one pass per frame)

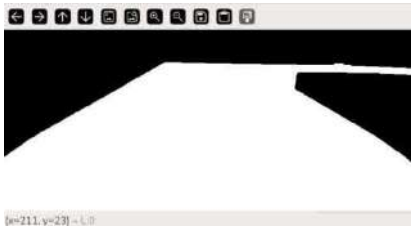
1. **Crop:** the 640×360 frame is cropped to the 640×280 region that actually sees the track.
2. **LAB conversion and colour masks:** the frame is converted to LAB once, and each colour is thresholded with `cv2.inRange`, then cleaned with a 5×5 morphological **open** (removes specks) and **close** (fills holes). The masks run in parallel on a **4-worker thread pool** while the main thread does the greyscale conversion, which helps keep the loop at ~150 ms.
3. **Paint markers white:** all marker masks are OR-ed into the greyscale image, so lines, pillars and the pink wall are never mistaken for black walls.
4. **Floor vs walls:** a greyscale threshold (100 for Open, 110 for Obstacle) splits floor from walls, followed by a morphological close.
5. **Track outline:** the largest floor shape is outlined and filled to make the **track mask**. Pillar, marker and pink masks are AND-ed with it (`on_track`), so a coloured object off the field can never steer the robot.



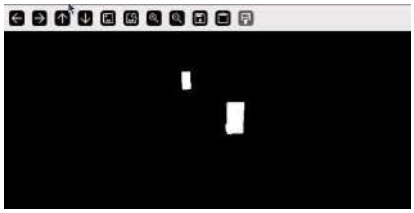
Open Challenge frame: corner lines + wall ROIs



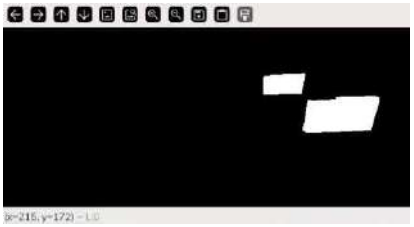
Obstacle Challenge frame: pillar aim line



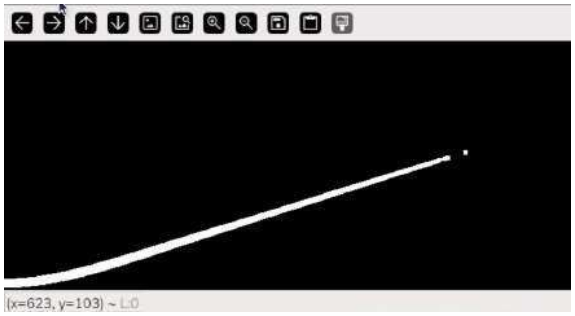
Track image



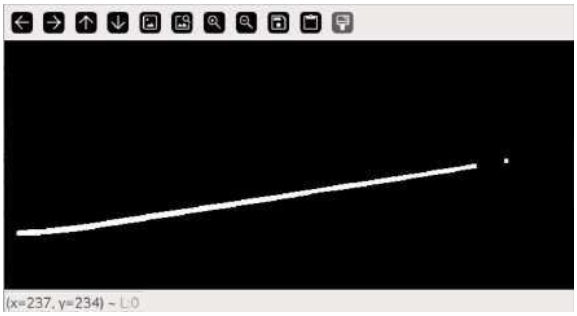
Obstacle mask



Parking block mask



Blue line mask



Orange line mask

Output	How it is built	Used by
Track image / mask	Greyscale threshold + track outline: walls black, floor white	Wall following (11.3), crash detection (11.5), on-track filtering
Blue / orange	LAB masks of the corner lines	Lap counter (11.6); start direction in the Open Challenge
Green / red	LAB masks of the pillars	Pillar avoidance (11.4), speed profile
Pink	LAB mask of the parking walls	Challenge detection (11.8), parking (11.7)

Colour	LAB lower [L, A, B]	LAB upper [L, A, B]
Blue	[22, 131, 75]	[95, 158, 112]
Orange	[78, 146, 159]	[139, 167, 194]
Green	[87, 65, 136]	[163, 112, 184]
Red	[61, 165, 140]	[128, 190, 171]
Pink	[35, 140, 115]	[179, 198, 139]

Why LAB, not HSV: LAB separates lightness (L) from colour (A, B), so thresholds on A and B stay consistent as brightness changes across the mat or between venues. HSV mixes hue with brightness near highlights and shadows, which in testing gave masks that flickered or dropped out under uneven lighting. LAB raised lap completion from 65% to 92% (11.10). Thresholds are tuned with our **LAB Adjuster** tool (`code/labadjuster.py`), which shows a live mask while sliders move the six L/A/B limits, so all five colours can be re-tuned on-field in under 5 minutes.

11.3 Wall Following

Obstacle Challenge (two ROIs). Two boxes (regions of interest) sit on the left and right of the frame. The side with more black wall pixels in its box is the side the robot is closer to. The difference between the two areas is the error fed to a **PD controller**:

```
area_left  = largest_wall_area(track_image, roi_left)
area_right = largest_wall_area(track_image, roi_right)

# opening boost: one wall vanishes -> act as if the other wall were huge
if area_left < WALL_OPENING_AREA and area_right > 100:
    area_right = area_right * OPENING_BOOST_MULT + OPENING_BOOST_ADD

wall_error = area_left - area_right
derivative = wall_error - prev_wall_error
control    = (wall_error * kp) + (derivative * kd)
angle      = STEER_CENTER - control
```

- **P term** steers harder the more unbalanced the two sides are; **D term** smooths frame-to-frame overcorrection.

- **No integral term, on purpose:** the error swings through zero constantly as the robot weaves, so an integral would wind up through tight corners and overcorrect when they open.
- **Opening boost:** without it, the error grows only gradually as the ROI slides past a gap, so the robot turns late. A collapsed wall area (< 700 px) only happens at a real opening, so inflating the opposite side (×2 + 25,000) produces an immediate, committed turn.
- **No wall on either side** means the robot is deep in a corner: it steers 35° toward the corner if a corner marker is visible, or 20° if not.
- **Tuning was empirical** on the physical track, because pixel area vs true distance depends on camera height and tilt (fixed first, 9.9). kp too high → side-to-side weaving on straights; kp too low → drifting into walls on tight corners. kd was raised until steering stopped chattering without dulling the reaction to a fast-approaching wall. Final values: kp = 0.09, kd = 0.0004 (damped near the inner wall: kp = 0.0018, kd = 0). The ROIs are tuned for a left run and mirrored for a right run.

Open Challenge (one ROI, new for V3). With no pillars, the robot only needs to hold a distance from the **inner wall**, which keeps its line short. It uses one ROI on the inner side and a PD loop that holds the wall area at a target (6,000 px). Two refinements made this reliable:

- **First-quarter lock-in:** the inner walls can be at different distances, so during the first quarter lap the robot holds whatever distance it started at (the first wall area it sees becomes the target) instead of swerving to a fixed one.
- **Corner boost:** when the wall area collapses by more than 25% in one frame (from at least 800 px), the robot is entering a corner. For the next 10 frames kp is multiplied by 2 and kd by 5, with steering capped at ±50, so the robot snaps into the turn instead of drifting wide.

11.4 Obstacle Avoidance

To steer around a pillar the robot needs the two sides of a right-angle triangle: the pillar's sideways offset from an aim point, and how far ahead it is. `atan2` gives the angle between them, and that angle becomes the steering angle. A fixed offset and a colour-specific gain bias it so that **green passes on the left and red passes on the right**.

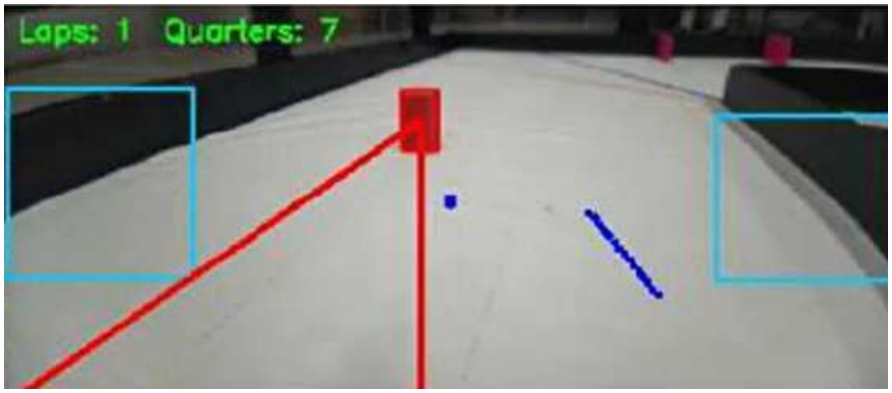


Figure 11.2 — A = aim point at the bottom of the frame, B = pillar centre; θ from atan2

```
target_x = GREEN_PILLAR_TARGET_X if is_green else RED_PILLAR_TARGET_X   # 590 / 50 px
horizontal_distance = x_center - target_x
vertical_distance   = FRAME_HEIGHT - y_center
pillar_angle_deg    = degrees(atan2(horizontal_distance, vertical_distance))

if is_green:   # pass on the left
    steer = STEER_CENTER - ((pillar_angle_deg + PILLAR_STEER_OFFSET_DEG) * GREEN_PILLAR_KP)
else:          # pass on the right
    steer = STEER_CENTER - ((pillar_angle_deg - PILLAR_STEER_OFFSET_DEG) * RED_PILLAR_KP)
```

- **Which pillar:** of all green and red blobs larger than 200 px, the one closest to the bottom-centre of the frame (the robot) is chosen.
- **Aim point:** the target x is offset toward the side the robot must end up on (50 px from the edge), so the angle already points to a clearance lane rather than at the pillar.

- **Three zones by height in the frame:** pillar above y = 150 px → steer by the angle above; between 150 and 265 px (close) → **hold the last angle**, because a pillar that close is partly out of frame and its centre jumps around; below 265 px → already passed, ignore.
- **After a pass:** for 15 frames steering is clipped to ±50 so the robot does not jerk back toward the wall.
- **Separate gains** (GREEN vs RED) exist because the two pass directions interact differently with the Ackermann turning circle, so each can be tuned on its own (both ended at 2.2 with a 45° offset).

PILLAR-AWARE VARIABLE SPEED (NEW FOR V3)

Speed is tied to the same pillar data. With no pillar in view the robot cruises at PWM 200. When a pillar is seen far away (above y = 150 px) it **boosts to 255**, because the far part of the approach is safe to cover quickly. Once the pillar enters the close zone it **slows by 4 PWM per frame** back toward cruise, so the critical part of the pass happens at a controlled speed. This is what took the Obstacle Challenge from ~70 s to ~50 s at the same ~95% accuracy (12.1).

11.5 Crash Detection

```
outer_crash = navigator.is_hitting_outer_wall(direction, track_image)
inner_crash = navigator.is_hitting_inner_wall(direction, track_image)  # only while a pillar is present
kp = AREA_WALL_KP_DAMPED if damped else profile.area_wall_kp
```

Check	How it is measured	Response	Clears when
Outer wall too close	One pixel straight ahead (centre column, 138 px up from the bottom) is black	Ignore all other steering; turn hard into the corner (highest priority); leaving the track ends the run	Pixel is floor again
Inner wall too close	15 sample points on a line on the inner side (from 50 px to 110 px up); any on a wall pixel	Skip pillar steering; wall following continues with damped gains (usually just a tight corner cut)	Line clear or pillar gone

Both checks **latch** into a small state (`WallState.NONE` / `INNER` / `OUTER`) until the condition clears. Without latching, a reading sitting on the threshold would snap the robot in and out of a hard recovery turn every other frame, which is far worse for stability than staying in recovery slightly longer. The inner check only runs while a pillar is pushing the robot toward the inner wall; otherwise wall following already handles it.

11.6 Lap Counter

The robot counts **quarter turns**, one per corner. Only the marker colour that belongs to the driving direction is counted (**blue for a left run, orange for a right run**), and only pixels inside the track mask, so the other colour and anything off the field are ignored.

- **Enter a turn:** marker area above 150 px for **3 consecutive frames**.
- **Finish the turn:** area below 60 px for several consecutive frames (Obstacle: 14 frames on a left run, 4 on a right run; Open: 2) **and** a minimum time in the turn, or a **2.0 s safety limit** if the marker never shrinks.
- **Cooldown** (0.85 s Obstacle, 1.25 s Open) stops the same marker being counted twice.
- After **12 quarter turns** (3 laps × 4 corners) the Open Challenge keeps wall following for 0.5 s so it finishes inside the start section, then stops; the Obstacle Challenge goes straight to parking.

Quarter turns finish every few seconds rather than every 20+ s, so a missed or stuck count shows up quickly. The multi-frame confirmation filters single-frame misfires from motion blur, pillar occlusion or reflections, accepting a small, fixed, predictable delay instead of occasional wrong counts.

11.7 Parking

Parking has three stages, all in the Obstacle Challenge: **leaving** the parking lot at the start, **approaching** the lot after three laps, and **reversing in**. Left and right runs use the same code; a right run is a mirror image with its own tuned values (`ParkProfile`).

1. LEAVING THE LOT (START OF RUN)

The robot detects the direction from which half of the frame has more open floor, sets full steering lock away from the wall, waits 0.5 s for the servo, drives **950 encoder ticks** forward at PWM 100, re-centres the steering and starts the laps.

2. APPROACHING THE PINK WALL

Steering priority while approaching is: **a pillar clearly nearer than the pink wall > the pink wall > damped wall following** (with the outer-wall crash check still active).

```
pillar_is_closer = (pillar_found is not None
                    and pillar_found[1] < PILLAR_TOO_CLOSE_Y          # not already passed
                    and (pink_y is None or pillar_found[1] > pink_y + PARK_PILLAR_MARGIN))
```

- A pillar must be at least 5 px lower in the frame than the pink wall to win; this margin stops the robot flip-flopping between the two targets. While parking, a pillar is always passed on the outer-wall side, whatever its colour.
- The pink steering angle is **smoothed** (new = 0.4 × previous + 0.6 × current) and **held for 4 frames** if the pink wall flickers out, so stray pixels cannot jerk the steering.
- The robot has arrived when the pink wall is low enough in the frame and within the x tolerance of its aim point; it then brakes.

3. REVERSING IN

1. Short straightening nudge, then drive forward along the **outer wall** for 2.5 s with a single-ROI PD that holds a target wall area.
2. Reverse almost straight until the pink wall becomes visible behind the robot.
3. A short forward **hop** (1,625 ticks left run / 1,950 right run) to line up.
4. Three reverse moves: **full lock → straight → full lock the other way** (950 / 675 / 1,000 ticks on a left run; 1,100 / 775 / 1,025 on a right run), with a servo settle pause before each.

Encoder ticks, not time: a timed move covers a different distance as the battery discharges or the surface changes, so a timed reverse would park differently each run. Counting ticks (the reason an encoder was a requirement in 10.1) covers the same distance every time; this is how we reach **±1 cm** parking accuracy. Letting pillars interrupt parking follows the arbitration logic: a pillar is a nearer-term problem than a parking spot that is not going anywhere.

11.8 The State Machine

An explicit state machine means each handler only reasons about its own part of the run ("drive laps" knows nothing about button debouncing). This keeps the code small enough to test in isolation and lets us re-run one state (for example parking) without driving a full lap first. The International version adds a **DETECT_CHALLENGE** state, so one program handles both challenges: the Obstacle Challenge starts inside the parking lot, so if at least 500 px of pink is in view at the start it is the Obstacle Challenge; if not, it is the Open Challenge.

```
def run(self):
    try:
        while self.state != RunState.FINISHED:
            self._handlers[self.state]()
    finally:
        self._cleanup()   # always brakes the motor and saves the video, even after a crash
```

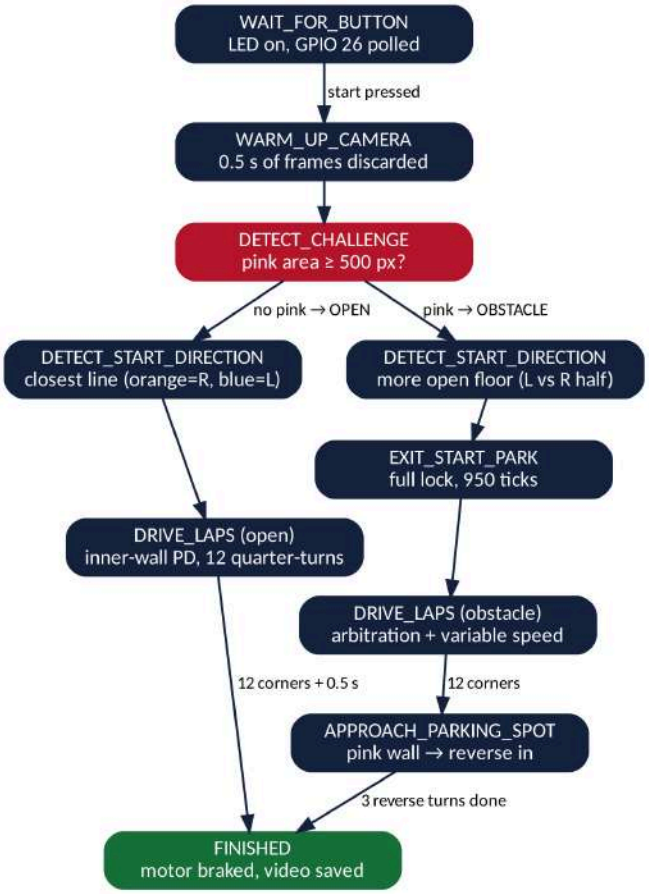


Figure 11.3 — International state machine (both challenges in one program)

Direction is measured once at the start and used by every later state, since clockwise and counter-clockwise runs need mirrored behaviour. In the Open Challenge it comes from whichever corner line is closer (orange = right, blue = left, defaulting to right after 1 s if no line is seen); in the Obstacle Challenge it comes from which half of the frame has more open floor.

11.9 Arbitration

```
pillar_angle = navigator.get_steer_angle(direction, green_on_track, red_on_track, track_image)
wall_angle   = navigator.area_wall_follow_angle(track_image, direction, blue_area, orange_area,
                                                damped=navigator.inner_wall_warning)
angle = pillar_angle if pillar_angle is not None else wall_angle
angle = navigator.clip_after_pillar(angle)
speed = navigator.update_drive_speed(speed)
```

Priority	Condition	Winning behaviour	Why it wins
1 (highest)	Outer-wall point is on a wall	Forced hard turn into the corner	Leaving the track ends the run
2	Inner wall close AND pillar nearby	Damped wall following (pillar steering skipped)	Steering hard for a pillar while tight to the inner wall risks turning into an outer-wall crash
3	Pillar detected, no crash active	Pillar avoidance (or hold last angle when close)	Hitting a pillar costs points and knocks the robot off its line
4 (default)	None of the above	Normal PD wall following	The steady-state fallback, not a competing priority

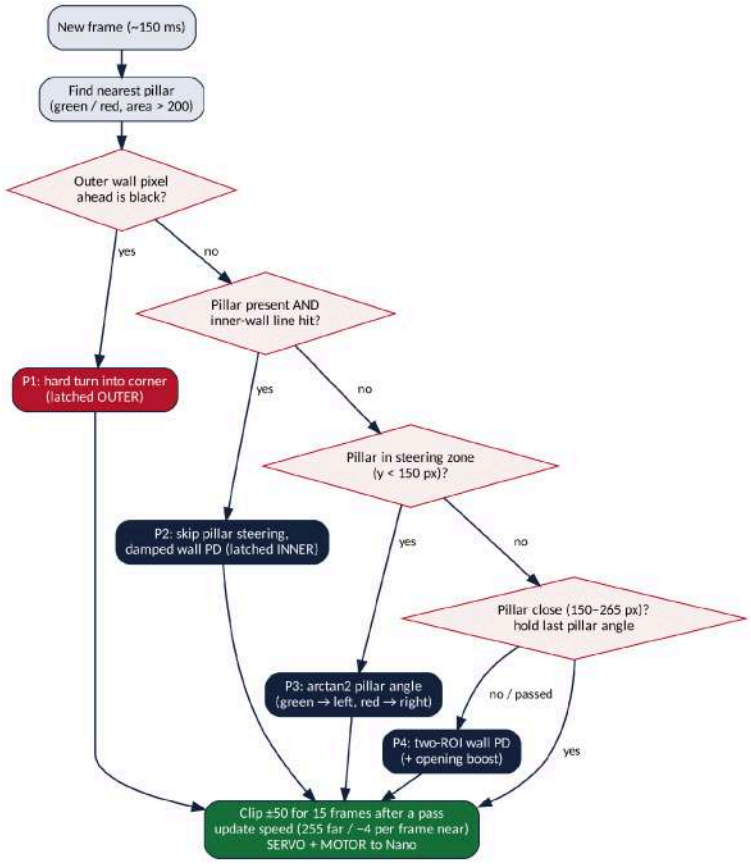


Figure 11.4 — Drive-laps decision flow for one frame (Obstacle Challenge)

Scenarios: mid-corner and slightly tight to the inner wall with no pillar → priorities 1 and 2 do not fire; wall following simply corrects. Same position with a pillar ahead → priority 2 fires and suppresses pillar steering, so the robot safely finishes the corner before dealing with the pillar on the next frames. A pillar passed → the 15-frame clip stops the robot swinging back too hard.

11.10 Software Testing & Iteration

Test stage	Method	Metric captured
Baseline run	5 laps, standard mat, unmodified code	Lap completion %, average lap time, parking accuracy %
Change applied	Same 5-lap protocol, one variable changed (e.g. HSV → LAB)	Same metrics, directly comparable
Latency check	Onboard timer logs per-frame time over 60 s	Average and maximum pipeline latency (ms)
Repeatability	3 repeat trials of the same configuration	Spread across trials
Random layouts	Layouts from the Random Combination Generator (7.3); every run recorded on video	Failures traced frame by frame on the recorded overlay

Merge rule: a change went into the final build only if it matched or improved the baseline on at least two of lap completion, parking accuracy or latency, without making the third more than 10% worse.

#	Change tested	Lap compl.	Parking acc.	Latency	Result	Notes
1	HSV thresholding	65%	70%	160 ms	**FAIL**	Inconsistent under overhead lighting
2	LAB colour space	92%	70%	155 ms	**PASS**	Adopted (11.2, 12.1)

#	Change tested	Lap compl.	Parking acc.	Latency	Result	Notes
3	Variable speed v1 (open-path based)	75%	70%	150 ms	**FAIL**	Faster but inconsistent run to run
4	Fixed speed	90%	85%	150 ms	**PASS**	Used at Nationals (V2)
5	Camera + ToF + gyro	88%	90%	600 ms	**FAIL**	Latency 4× over budget
6	Camera-only	90%	85%	300 ms	**PASS**	Adopted (10.6, 12.1)
7	Single board (Pi only)	78%	85%	300 ms*	**FAIL**	*Vision fine; PWM jitter caused steering lag
8	Pi + Arduino split	90%	90%	150 ms	**PASS**	Adopted (11.1, 12.1)
9	Variable speed v2 (pillar-aware)	~95%	±1 cm	~150 ms	**PASS**	Adopted for Internationals: ~70 s → ~50 s

- **Lap completion:** ~65% (HSV baseline) → ~90% (LAB + camera-only + split board) → **~95%** over 15 consecutive laps on the International build.
- **Parking accuracy: ±1 cm** final, against 60–70% success in early and failed configurations.
- **Latency:** held at ~150 ms; peaked at ~600 ms with ToF/gyro fusion, a 4× regression that justified camera-only.
- **Speed vs consistency:** our first variable-speed attempt (#3) changed speed based on the open path ahead and behaved differently from run to run depending on exactly when the transition happened, so Nationals ran at fixed speed (~95%, ~70 s). The second version (#9) ties speed to the pillar's position instead, which is predictable, and gave **~95% at ~50 s**.

11.11 Performance & Validation

Test	Target	Achieved (V3)	Margin
Lap time (Open Challenge)	< 45 s	**24 s**	+8 s buffer
Lap time (Obstacle Challenge)	< 90 s	**50 s**	+11 s buffer
Parking accuracy	±3 cm	**±1 cm**	Exceeded
Power runtime	> 60 min	**75 min**	Exceeded
Processing latency	< 200 ms	**~150 ms**	Headroom

Why the buffers: times change from run to run with the randomised layout (direction, pillar combination, parking-lot position), so each buffer is the margin we keep to the target across layout combinations, not just the gap to our single best run.

Validation: tested under 3 different lighting conditions · **15 consecutive laps at a 95% success rate** · green and red pillars tested at varying distances (20, 50 and 80 cm). Run videos: **Open Challenge** youtu.be/3cM_T3cttLk · **Obstacle Challenge** youtu.be/weySsihPY30

12.0 Systems Thinking & Engineering Decisions

Building this robot meant constantly balancing four connected subsystems (power, sensors, software and mobility), where a change in one almost always affected the others. Many of our choices only make sense as trade-offs made to satisfy several competing constraints at once. Moving the camera, for example, changed the vision tuning; isolating the servo changed the PCB layout and the second plate. This section covers the key decisions, the alternatives we considered, the hard constraints that shaped the design, a structured risk analysis, our failures, and how the robot changed from V1 to V2 to V3.

12.1 Trade-Offs

Decision	Reasoning	Evidence
Pi + Nano split (vs one board)	The Pi is not slowed by motor loops, so vision runs faster; the Nano handles timing exactly when needed, independent of the Pi's scheduling; each board does one job well; debugging is easier (you know which board to check)	Pi-only: 78% laps, PWM jitter. Split: 90% laps, 150 ms
No extra distance / gyro sensors	(1) Software: more data to read and fuse each loop slows control. (2) Practical: more wiring, failure points and calibration; every extra wire is a chance for a loose connector or a drifting sensor between runs. Also a deliberate challenge to ourselves to go camera-only.	~150 ms → ~600 ms with ToF + gyro (+450 ms, ≈4×)
Variable speed (vs fixed)	The International profile accelerates when a pillar is detected far away, then slows down step by step once it is close, for precise manoeuvring. Accuracy is the same as fixed speed, but the run is much faster: the slow-down keeps the critical part of the path controlled while the rest of the run uses the higher speed.	Fixed: ~95%, ~70 s. Variable: ~95%, ~50 s
LAB (vs HSV)	Noticeably better detection and accuracy under our lighting, worth it even though HSV is more common	65% → 92% lap completion
Battery at plate centre	The heaviest single component, low and centred on the bottom plate, spreads weight evenly front-to-back and side-to-side, so the robot is not nose-heavy, tail-heavy or lopsided. This improves stability and predictability in turns.	Fixes VHR's tipping CG (7.1); no tipping at 0.598 m/s
Lower, more rearward camera (V3)	Less off-field noise and longer pillar visibility, at the cost of re-tuning ROIs and pillar thresholds	~15% less off-field view; +340 ms pillar visibility

Speed mode	Accuracy	Obstacle Challenge time
Fixed speed (V2 Nationals)	~95%	~70 s
Variable speed (V3 Internationals)	**~95%**	**~50 s**

12.2 Constraints

- **No differential (tank) drive allowed:** front steering was mandatory from the start. Front steering ties turning radius to chassis length ($R = L / \sin\theta$), and the outer radius grows with the robot's width ($R_{outer} = R_{centre} + W/2$). The drivetrain, motor and steering had to be packed tightly back to back to keep the radius manageable for parking, so a rule about *drive type* ended up dictating chassis layout and packaging density.
- **1.5 kg weight cap:** weight *distribution* mattered as much as the total. Every gram competes for the same budget, which forced trade-offs between convenience (easy battery access) and performance (low, centred mass); performance won.
- **Power budget tied to runtime:** the robot had to run for over an hour on one charge. Every power-hungry addition has a knock-on effect (more draw → bigger, heavier battery → slower robot), which quietly ruled out components that looked good on their own.
- **Control-loop speed:** sense-think-act must react before the robot reaches a wall or pillar. This is why processing was split across two boards, why masks are computed in parallel, and why anything that added latency without a matching benefit (extra sensors, heavier vision models) was ruled out.

12.3 Risks & Failures

SYSTEM FAILURE MODE AND EFFECTS ANALYSIS (FMEA)

To evaluate reliability systematically we built an FMEA of our main hardware, electrical and perception risks. Each is scored 1–10 for **Severity (S)**, **Occurrence (O)** and **Detection (D)**, giving a **Risk Priority Number RPN = S × O × D**. Every entry below is a failure we actually experienced; the detailed stories follow.

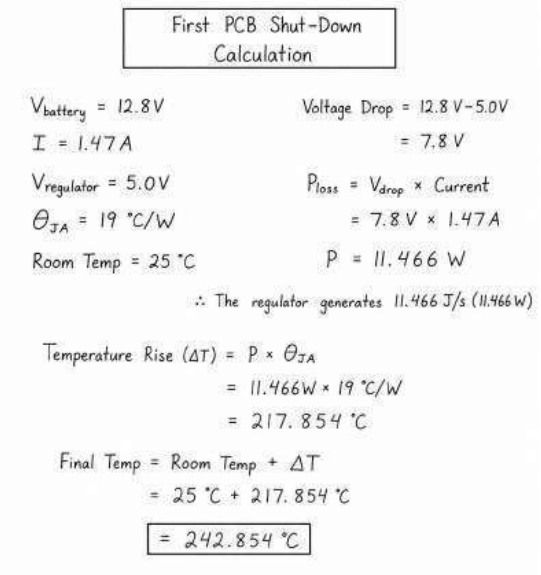
Subsystem	Failure mode	Potential effect	S	O	D	RPN	Mitigation implemented
Actuation (servo)	MG90S servo under high steering load	Stalling, 3.0–5.0 A spikes, brownout and MCU reset	7	6	4	**168**	9g coreless metal-gear servo (4 kg·cm) on an independent 8.4 V 8 A UBEC
Power (PCB V1)	7805 linear regulator dropping 7.8 V at ~2 A	Thermal runaway; ~175 °C junction; thermal shutdown	8	9	2	**144**	85–95% efficient 5 V DC-DC buck converter
Perception (vision)	Camera exposure shift under arena lighting	Colour-threshold failure; lines and pillars lost	8	5	3	**120**	HSV ⟶ LAB colour space; on-field LAB re-tuning
Electrical (driver)	DRV8871 VM pin tied to logic/servo lines	12.8 V delivered to 5 V parts; driver destroyed	9	4	3	**108**	Corrected routing; motor power isolated from logic; pinouts verified against datasheets
Mobility (steering)	Parallel steering geometry	~12° slip at full lock; unpredictable cornering	6	7	2	**84**	100% Ackermann linkage; slip < 2°

12.3.1 PCB Version 1: Thermal Failure

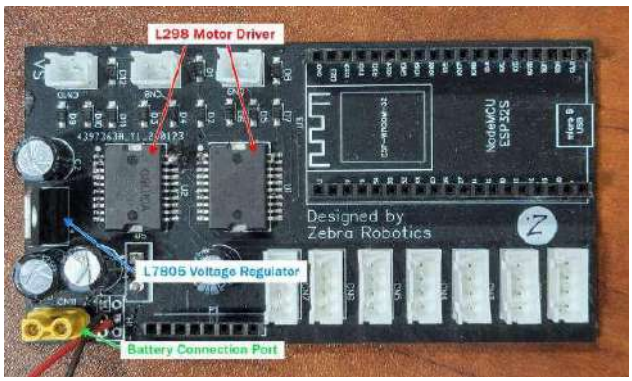
Symptom: after several test runs the robot always shut down after about three corner turns. The pattern repeated, so it pointed to a systematic problem rather than a random fault. The L298 driver and 7805 regulator were extremely hot to the touch. **Diagnosis:** the 7805's datasheet showed it was reaching ~150–175 °C, above its rated maximum junction temperature, which triggers its built-in thermal shutdown. That explains why the robot recovered after cooling down.

Why (power dissipation): a linear regulator burns off the input–output voltage difference as heat at whatever current flows. With a 7.8 V drop (12.8 V ⟶ 5 V), the worst case at ~2 A is **15.6 W**, far more than a small TO-220 package can shed in free air without a heatsink. Using the current we actually measured (1.47 A), the regulator dissipates 11.47 W. With its junction-to-ambient thermal resistance of 19 °C/W, that is a rise of 217.85 °C above a 25 °C room, giving a predicted temperature of **242.85 °C**. Thermal shutdown was mathematically guaranteed under our load, which matched exactly what we saw on the mat.

> **Note**
> Junction temperature is how hot the chip itself gets while running. Junction-to-ambient thermal resistance is how many degrees hotter than the surrounding air it gets for each watt it produces. The regulator could not get rid of its heat fast enough, so it overheated within a few corners.



First PCB shut-down calculation



PCB V1: L298 driver and L7805 regulator

PCB V1 load (5 V rail)	Voltage	Normal current	Peak current	Power
ESP32	5 V	0.15 A	0.30 A	0.75 W
Servo port (MG90S)	5 V	0.15 A	0.70 A	0.75 W
Motor encoder port	5 V	0.02 A	0.03 A	0.10 W
Total		**0.32 A**	**1.03 A**	**1.60 W**

The datasheet peak total (1.03 A) is lower than the **1.47 A we measured**. Datasheet peaks assume one component at a time, while the multimeter captured the ESP32, servo and encoder drawing at the same time in a corner turn. We used the measured value in the calculation because it reflects real operating conditions.

Temporary fix. *Hypothesis:* more surface area should shed heat faster and delay shutdown. *Action:* an aluminium VEX piece attached to the 7805 as a heatsink. *Result:* runtime rose from 1½ to 2¼ laps, a **50% improvement**, confirming that heat (not wiring) was the root cause. But the regulator still shut down eventually, because a small passive heatsink without forced airflow cannot carry away that much heat indefinitely.

Root cause and decision. With layout, wiring and component defects ruled out, a linear regulator was simply the wrong tool for a 7.8 V drop at this current. Its efficiency is roughly Vout/Vin (5 / 12.8 ≈ **39%**), so over 60% of the 5 V-rail power became heat regardless of cooling. We switched to a **buck converter**, which stores energy in an inductor while switching rapidly and reaches **85–95%** efficiency. That is the 5 V buck in our final architecture (10.4).

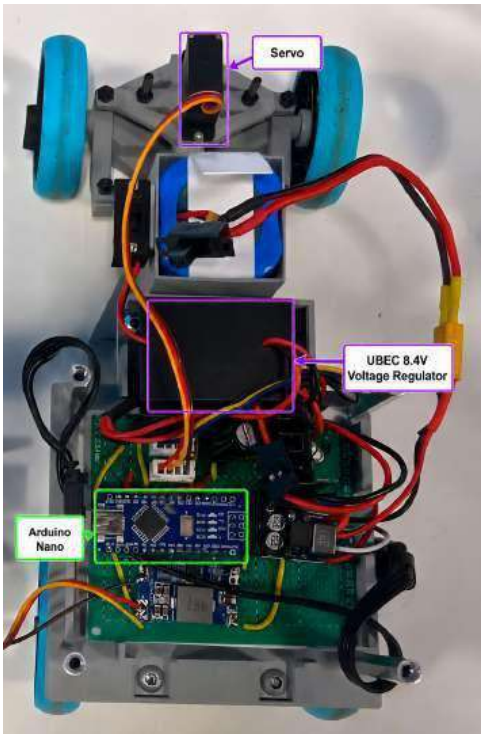
12.3.2 PCB Version 2: Motor Driver & Servo Failure

Problem	Root cause	Fix
DRV8871 would not reliably run the motor	**Signal mismatch:** the ESP32 outputs 0–3.3 V PWM; the DRV8871 needs 5 V PWM to switch reliably	Switched ESP32 → Arduino Nano (full 5 V PWM)
Plugging in the servo shut down the ESP32, driver and all electronics	**No power isolation:** servo current spikes pulled down the shared rail	Dedicated 7805 on the servo line, with a heatsink
MG90S servo burned out; servo 7805 still overheating	Servo under-rated for the load (plastic gears, brushed motor, near stall); large voltage drop across the regulator	9g coreless full-metal-gear servo (4 kg·cm, 4.8–8.4 V) + dedicated 2–8S UBEC-8A set to 8.4 V

Why this servo: (1) **torque headroom**: 4 kg·cm keeps it well away from stall, the condition that destroyed the original; (2) **full metal gears**: the failure was mechanical as well as electrical, and metal resists the stripping plastic gears suffer under repeated hard corrections; (3) **4.8–8.4 V with a coreless motor**: higher voltage gives more torque and speed without changing the control signal, and a coreless motor responds faster and more efficiently than a brushed motor of the same size.

Why this UBEC: (1) **isolated, dedicated supply**: the servo gets its own regulated rail, which directly solves the brownout; (2) **matched output**: adjustable 5.2 / 6.0 / 7.4 / 8.4 V, set to exactly 8.4 V for full rated torque; (3) **current headroom**: 8 A continuous against the servo's 3.0–5.0 A peak, so spikes do not sag the voltage; (4) **wide input (2–8S)**: it accepts our 12.8 V 4S pack well within range. **Result:** stable, high-current steering power independent of the other electronics, and a servo that steers without stalling or overheating.

> **VM pin wiring failure**
>
> While wiring the DRV8871 we powered the Pi, servo and Nano from the driver's **VM pin**, not realising it is internally tied to P+, which connects directly to the battery. Components expecting a regulated 5 V (or the servo's rated input) received the full 12.8 V, which overloaded and blew the DRV8871.
>
> **Lesson:** VM is a motor power input, never a general-purpose output. Every logic component and the servo needs its own regulated supply, and we now check every pin against the datasheet before powering a new board.



Final PCB with the implemented changes (Nano, isolated UBEC, servo port)

12.3.3 Risk: Motor Adapter

The motor adapter was slightly too long for the drivetrain clearance and pushed the small differential gear forward along its shaft, binding it against the housing. We saw visible scratching, occasional jamming, and wheels that caught or slowed now and then. **Fix:** shortening the adapter by **1.0 mm** let the gear sit correctly while still coupling the D-shaft reliably; afterwards the wheels turned smoothly under load with no more scratching or jamming. **Follow-up for V3:** this showed how sensitive the gear mesh was to the alignment of separate parts, which was one of the reasons we combined the gear holder and axle supports into a single rear axle casing (9.5).

12.4 Iterations

The robot went through three major versions. Each was built to answer specific questions, and each version's failures became the next version's requirements.

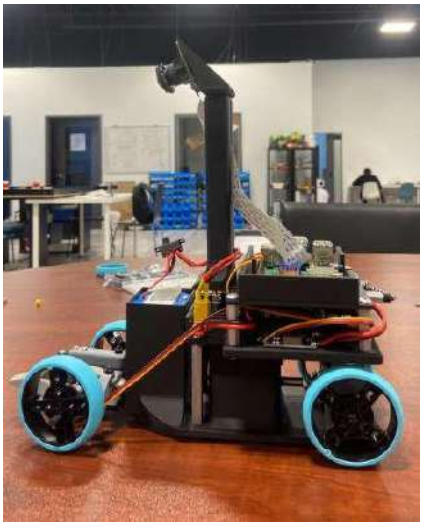
Version	Purpose	Outcome
V1 — Prototype	Test our core assumptions (chassis layout, wheels, steering geometry, drivetrain) on a real track instead of relying on CAD	Found misaligned axles, non-Ackermann steering, a weak motor adapter and an overheating PCB
V2 — Nationals robot	A complete, reliable competition robot: Ackermann, metal drivetrain, PCB V3, camera-only LAB vision, fixed speed	Competed at Nationals; ~95% accuracy, ~70 s Obstacle runs; showed where time and consistency were lost
V3 — Internationals robot	Keep the V2 architecture and fix what V2 showed: camera position, drivetrain alignment, Pi protection, steering range, speed, code structure	~95% accuracy at ~50 s Obstacle runs and 24 s Open runs

12.4.1 V1 — Prototype

Before the competition design we built a **V1 prototype**, not for performance but to test our core assumptions on a real track and find failure points while they were still cheap to fix.



Part iterations: first plate, steering, second and third plates



V1 prototype

System	Tested	Failure observed	Root cause	Fix applied (V2)
Chassis	Monocoque layout	Front/rear wheels misaligned, robot angled up	Servo holder not high enough	Enlarged chassis near the servo holder to raise the front wheels
Steering	Ackermann	Steering not sharp; tyres skidding	Geometry not calculated; wheels not at pivot point	Calculated 12° Ackermann; wheels at the pivots
Differential	Motor adapter & diff smoothness	Adapter weak, breaking; small gear scratching big gear	Adapter 1–2 mm off size, gear at an angle; weak print	Fixed measurements; printed in stronger material (PETG, solid)
Electronics	All components together	Driver and regulator overheating	Regulator could not handle the battery load	New custom PCB with better parts (PCB V2 → V3)

12.4.2 V2 — Nationals Robot

V2 was our National competition robot: the first version with every final subsystem in place (modular three-plate chassis, 100% Ackermann, metal differential and stainless-steel axles, PCB V3 with isolated rails, camera-only LAB vision on the Pi + Nano split). It met our consistency goal at **~95% accuracy**. Analysing its runs, our test videos and the build itself showed where it was still losing time and margin, and those observations became the V3 requirements.

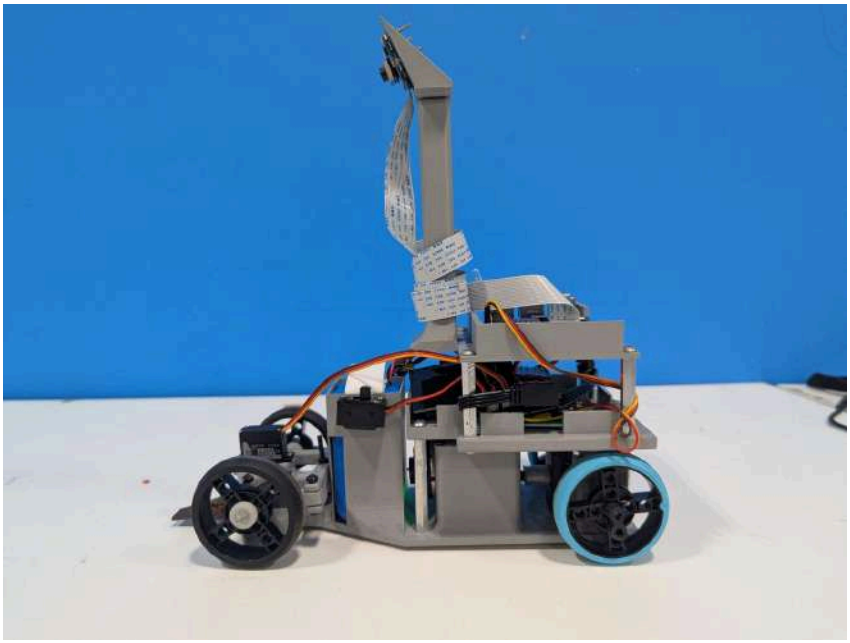
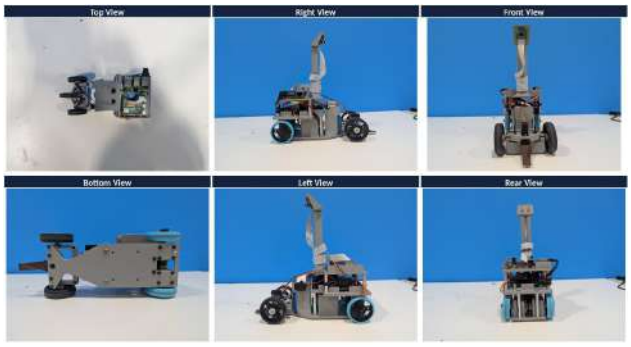
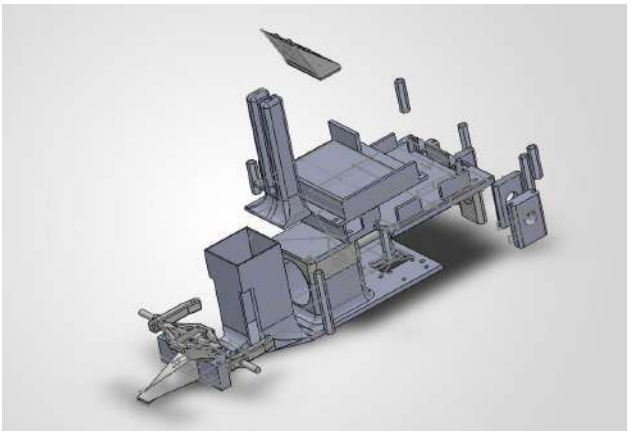


Figure 12.1 — V2 Nationals robot, left side



V2 Nationals robot: six views



V2 Nationals: exploded CAD

System	Tested (V2)	Issue observed	Root cause	Fix applied (V3 Internationals)
Camera height	Lens on a tall stand (~267.5 mm)	Colours beyond the field border appearing in the masks (noise)	High lens + wide FOV sees past the walls	Lowered **37.5 mm** to 230 mm → ~15% less beyond-field view
Camera position	Camera further forward	Pillar left the view before the body had passed it; risk of clipping the next pillar	Short look-back once the pillar is alongside	Moved **18.3 mm rearward** (58.8% from nose) → pillar visible **~340 ms longer**
Rear drivetrain	Gear holder + two separate axle supports	Gear mesh sensitive to part alignment (see 12.3.3); extra friction	Several separate parts aligned by hand	Single **rear axle casing** for axles, differential and motor adapter
Pi holder	Open holder; serial cable exiting sideways	Board exposed; cables sticking out where they could catch walls or pillars	Layout of the third plate	**Enclosed Pi case**, mounted lengthwise, serial port facing rearward; tidier cable routing
Steering range	Centre 97, ±43 servo units	Wider turns than wanted in corners and during parking	Linkage parts and calibration limited usable lock	Shortened steering T, gapped cross link and servo holder; centre 105, **±60** units
Speed	Fixed PWM 200	~70 s Obstacle runs; earlier variable speed was inconsistent	Speed changes not linked to the real hazard (pillars)	**Pillar-aware variable speed**: 255 far, −4/frame near, 200 cruise → **~50 s**

System	Tested (V2)	Issue observed	Root cause	Fix applied (V3 Internationals)
Pillar tuning	Steer zone 60 px, kp 1.45, aim 15 px from edge	Late, sharp corrections around pillars	Pillars tracked only when already close	Steer zone **130 px**, kp **2.2**, aim **50 px**, 15-frame post-pass clip
Software	Two scripts (1,692 lines), chosen by hand	Duplicated values; risk of running the wrong script	No shared structure between challenges	**One modular package**, challenge auto-detected from pink, profiles, video logging

How these changes became the International robot. None of the V3 changes replaced the V2 architecture; they refined it. The mechanical changes (camera, casing, Pi case, steering) gave the software a cleaner image, more time to react to each pillar and a tighter turning circle. The software changes then used that margin to go faster: because pillars are seen earlier and longer, the robot can boost to full speed on the approach and slow only for the pass itself. Accuracy stayed at ~95% while the Obstacle Challenge time fell by about 20 s, which is exactly the "consistency first, then speed" order we set at the start of the season.

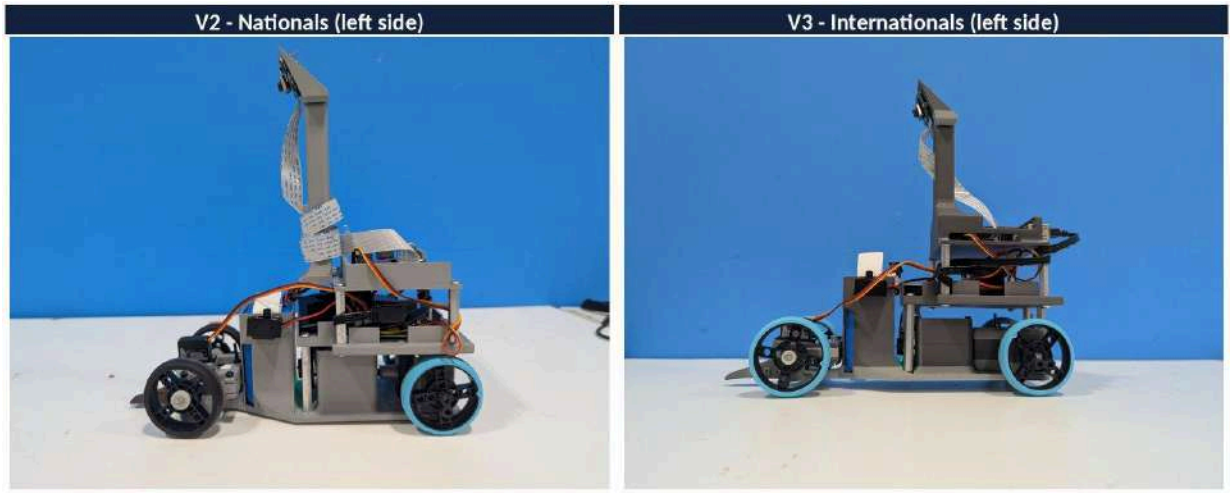


Figure 12.2 — V2 Nationals vs V3 Internationals, left side: lower, more rearward camera; enclosed Pi; rear axle casing

12.5 Safety, Reliability & Known Limitations

Hardware Safeguards

- **Battery isolation:** a dedicated compartment stops the pack shifting in impacts.
- **Voltage regulation:** independent rails (12.8 V motor/driver, 5 V logic, 8.4 V servo) stop servo current spikes browning out the logic.
- **Modular design:** the three-layer chassis lets any damaged section be replaced without rewiring the whole robot.
- **Cable management:** shielded connectors and routed wiring prevent accidental disconnects in impacts; on V3 the enclosed Pi case keeps cables tucked in.
- **Thermal:** aluminium heatsinks on the DRV8871 and the 5 V buck; an 85–95% buck chosen over a 39% linear regulator specifically to remove the thermal-shutdown risk; the robot runs cooler in competition than in extended testing.
- **Diagnostics:** status LED shows ready / running / error; live serial debug output and on-video overlays (lap count, timer, speed) during testing.

Software Safeguards

- **Encoder validation:** movement is checked against the expected displacement; a `MOVE` that does not finish returns `DONE,...,TIMEOUT` and mismatches are logged.
- **Wall proximity checks:** continuous inner/outer wall scanning stops the robot driving into barriers.
- **Speed limiting:** speed is capped so the robot can always react within the ~150 ms vision cycle.

- **Guaranteed cleanup:** the state machine always brakes the motor, stops the camera and saves the video, even if the program crashes.

Competition-Day Checklist

✓	Check	✓	Check
☐	Battery fully charged (12.8 V nominal, > 12.0 V at start)	☐	Steering servo moves smoothly across full range
☐	Servo is turned on	☐	Camera lens clean; focus sharp
☐	All connectors seated; no loose wiring	☐	All LEDs light on power-up
☐	All 3D-printed parts checked for cracks	☐	Test run on practice mat: all systems responsive
☐	Wheels spin freely; differential not binding	☐	Backup servo and PCB in the pit area

Known Limitations & Mitigation

Issue	Mitigation
Battery voltage drop under heavy load	2000 mAh pack with low internal resistance; runtime validated (75 min) before competition
Servo stalling under steering load	High-torque (4 kg·cm) servo on a dedicated UBEC prevents brownout; tested extensively
Colour detection inconsistency at a new venue	LAB colour space (vs HSV); thresholds re-tuned on-field with the LAB Adjuster if needed
Motor encoder drift	PD control with derivative smoothing reduces noise; timeout fallback if the encoder fails

12.6 Testing & Validation for Reliability

Category	Tests performed
Stress	**Battery discharge:** 15 consecutive laps, performance maintained through depletion · **Thermal:** extended runs in warm environments, no shutdown · **Mechanical:** full-lock steering and obstacle impacts, no failures
Environmental	**Lighting:** colour detection validated under fluorescent, LED and natural light · **Surface:** smooth and textured mats, predictable steering · **Electrical noise:** Wi-Fi/EM interference does not affect motor or servo control
Redundancy & fallback	**Dual PCB:** a second board is a drop-in replacement (the second plate fits both) · **Encoder fallback:** fixed-time moves if feedback becomes unreliable · **Recalibration:** colour parameters re-tuned on-field in under 5 minutes, no hardware changes
Layout coverage	Random Combination Generator layouts in both directions; each run recorded for frame-by-frame review

13.0 Reproducibility & GitHub

This section explains how to rebuild the robot from scratch (mechanical models, custom PCB, parts list and software) alongside our GitHub repository, **github.com/JumpyTruck/WRO-FE-2026-Sin-Theta**. Times are estimates from our own builds.

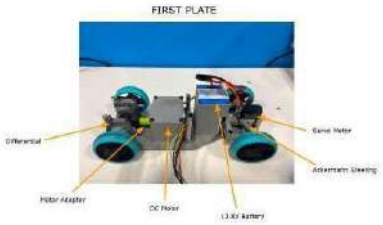
Phase	What you do	Time
1. Mechanical	Print parts, assemble chassis, rear axle and steering	4–6 h (+ ~24 h printing)
2. Electrical	Build and mount the PCB, wire everything	3–4 h

Phase	What you do	Time
3. Software	Set up the Pi, flash the Nano, tune colours	1–2 h
4. First test	Staged tests on a practice mat	30 min

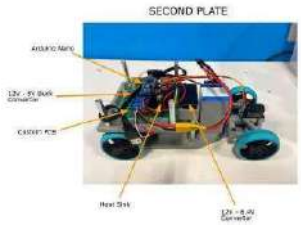
13.1 How to Assemble the Robot

Step 1: Print parts. Download the STLs from `models/CAD 3D Printed Parts/` (V3 parts are in `Inters Robot Files/`). Bambu Lab X1 Carbon or equivalent FDM printer, PLA, 15% gyroid infill, 5 walls, 0.2 mm layers (motor adapter: PETG, 100%). Quality check: no layer separation and all mounting holes aligned. **Step 2: Assemble the chassis** (≈45 min; M3 bolts and nuts, screwdriver, 2.5 mm Allen key, sandpaper for print edges). Order: base plate → battery mount → motor clamp → rear axle and steering → second story → third story.

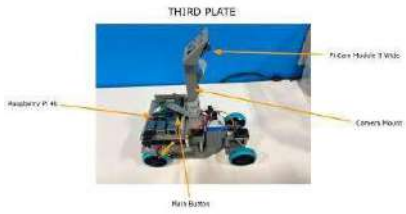
Part	File (Inters Robot Files)	Description
Base plate	`Chassis - Final Inters`	Steering holder section, low battery holder, mid-rear motor mount (upper/lower halves clamp the motor), rear axle casing mounts
Motor mount, top half / cover	`Top Motor Holder` · `Motor Cover Top`	Clamps over the motor and bolts into the lower half
Rear axle casing	`Axle Housing Bottom - Tall` + top	Holds axles, differential and motor adapter in one aligned housing
Second story (PCB holder)	`PCB Holder`	Main pocket seats the PCB; smaller pocket holds the servo UBEC (also fits the old PCB as backup)
Third story (Pi case + tower)	`Pi Reverse Gap`	Enclosed Pi case, mounted lengthwise with the serial port rearward; integrated camera tower
Camera mount	`Camera Mount - With Screw Holes - Final`	Standalone piece on the tower so the angle can change without touching the tower. Set to 155°
Steering parts	`Ackerman Steering Holder`, `Servo Holder Gap Final`, `Steering T - Short`, `Steering Cross Link - Gap`, knuckles	See below
Bumper	`Front Bumper`	Nose bumper / wing, attached with thick tape



First plate installed



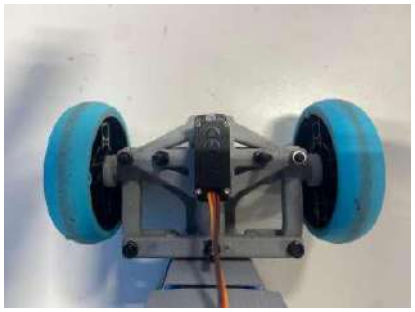
Second plate installed



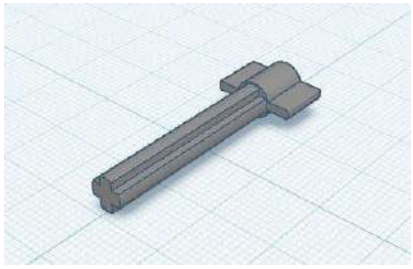
Third plate installed

Step 3: Install wheels and differential (≈30 min). Seat the metal differential and bearings in the rear axle casing with the motor adapter, insert the stainless-steel axles, add VEX spacers and attach the SPIKE Prime wheels. *Validation:* wheels spin freely with no friction; the differential engages smoothly.

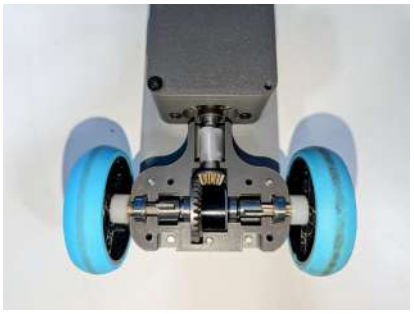
Step 4: Install the steering assembly (≈20 min). Mount the servo **inverted** in the steering holder so its output lines up with the knuckles; fit the knuckles (fixed at ~12° Ackermann) with nylon spacers; connect the steering T to the servo and the cross link to both knuckles. *Validation:* steering moves smoothly from full lock left to full lock right, with no binding.



Full steering assembly: installed



Stainless-steel axle (CAD)



Rear axle assembly: installed

Mechanical checkpoint

☐ All wheels spin freely ☐ Steering moves smoothly across full range ☐ Differential lets wheel speeds vary ☐ No printed parts cracked or stress-marked ☐ Assembly rigid, no wobble at joints

13.2 How to Build the Custom PCB

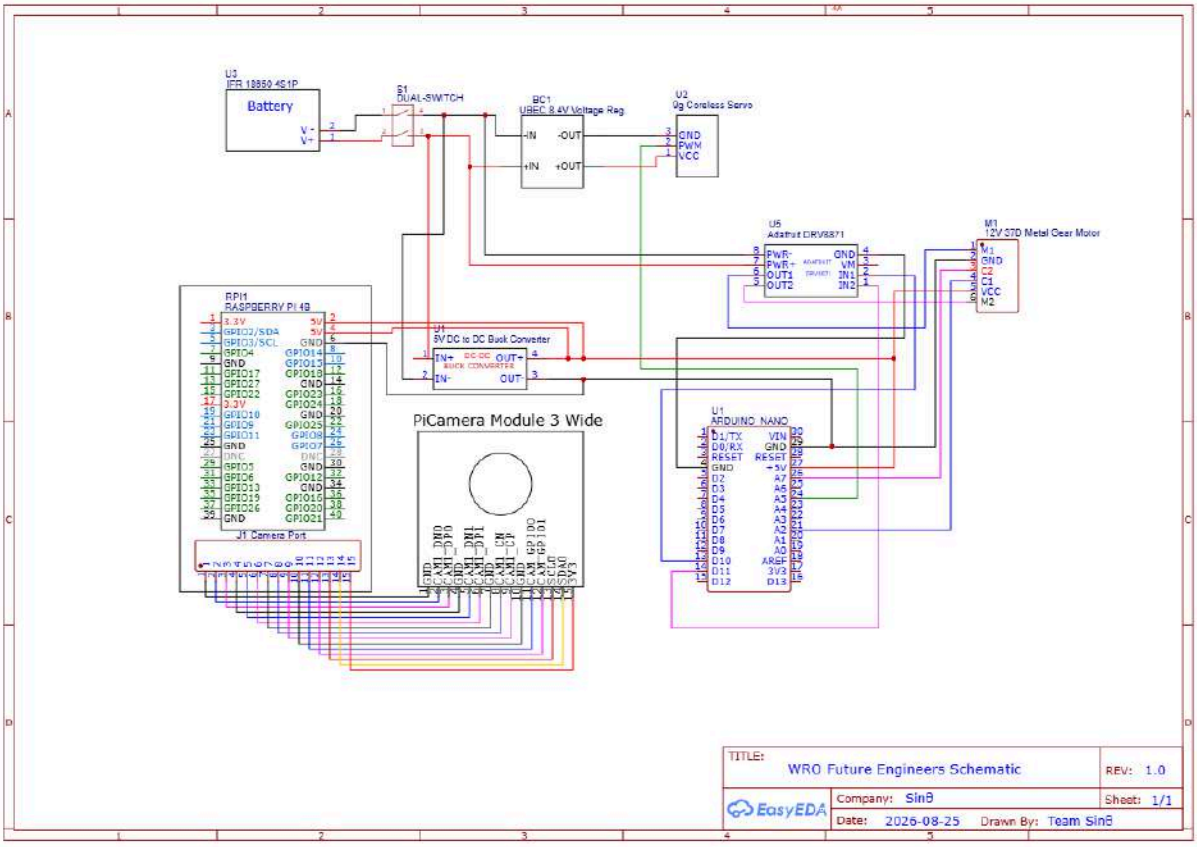


Figure 13.1 — PCB schematic: complete electrical architecture (electrical/Schematic_WRO-SinΘ-Final-Schematics)

Step	Actions	Time
1. Gather components	See the BOM (5.1); verify every item has arrived	—
2. Battery connections	Solder battery leads to a switch; solder switch +/− to the PCB	20 min
3. Power regulators	Mount the 5 V buck; battery +/− → buck input +/−; buck output + → 5 V rail, − → GND rail. Mount the 8.4 V UBEC; battery +/− → UBEC input; UBEC output +/− → servo VCC/GND	60 min

Step	Actions	Time
4. Motor & servo	DRV8871 power +/− → battery; OUT1/OUT2 → motor M1/M2; IN1/IN2 → Nano D10/D11; GND → Nano GND. Servo PWM → Nano D5. Encoder 5 V, GND, pulse → Nano (D2)	80 min
5. Pi & Nano	Power both from the 5 V buck; Pi ↔ Nano by USB serial; Camera 3 ribbon → Pi CSI connector	15 min
6. Mount in chassis	Battery in the base-plate holder; PCB on the second story; Pi in the third-story case; camera on the tower at 155°; check for loose wires	10 min

Electrical checkpoint

☐ Battery reads 12.8 V ☐ 5 V rail reads 5.0 V ± 0.1 V ☐ 8.4 V servo rail reads 8.4 V ± 0.2 V ☐ All GND points 0 Ω apart ☐ Motor driver responds to PWM ☐ Servo moves smoothly (Nano test code) ☐ Camera captures frames

13.3 Software Setup

You need: a Raspberry Pi (4 or 5) with Raspberry Pi OS; the camera on the CSI port; the Arduino Nano on USB; the start button and status LED on GPIO 26 and 16 (BCM). Full guide: `code/md/raspberrypi_setup.md`.

```
# 1. Update, enable the camera (Interface Options -> Camera -> Enable), reboot
sudo apt update && sudo apt full-upgrade -y && sudo reboot
sudo raspi-config

# 2. Find the Nano's port (plug/unplug and compare; usually /dev/ttyUSB0 or /dev/ttyACM0)
ls /dev/tty*                      # update SERIAL_PORT in utils/constants.py if needed
sudo usermod -a -G dialout $USER && sudo reboot

# 3. System packages (opencv / picamera2 rely on hardware libraries pip struggles with)
sudo apt install -y python3-pip python3-venv python3-opencv python3-picamera2 git

# 4. Virtual environment with system packages, then Python dependencies
python3 -m venv --system-site-packages venv
source venv/bin/activate          # re-run in every new terminal
pip install pyserial numpy gpiozero RPi.GPIO

# 5. Get the code
git clone https://github.com/JumpyTruck/WRO-FE-2026-Sin-Theta.git

# 6. Run: one program for BOTH challenges (it detects Open vs Obstacle by itself)
cd WRO-FE-2026-Sin-Theta/code/internationals && python3 main.py
```

Package	Purpose	Package	Purpose
opencv (system)	Colour masks, contours, steering maths	pyserial	Talking to the Nano over USB
picamera2 (system)	Reading frames from the Pi camera	RPi.GPIO	Reading the start button
numpy	Array maths throughout the pipeline	gpiozero	Controlling the status LED

Arduino firmware (15 min): upload `code/nationals/Arduino_Serial.ino` to the Nano, then check Pi–Nano communication with `code/Serial Test Code/`. **Colour tuning (20–30 min):** run `code/labadjuster.py`; for each colour (blue, green, red, orange, pink) show the real line or pillar and adjust the LAB thresholds until detection is reliable across positions and lighting, then save them to `utils/constants.py` and verify on a practice mat.

Software checkpoint

☐ Pi boots and runs Python scripts ☐ Nano receives and executes motor commands ☐ Pi–Nano serial reliable (no dropped packets) ☐ Camera frames clear, correct white balance ☐ All 5 colours detected reliably ☐ No syntax errors or import failures

13.4 First Test Run on Practice Mat (30 min)

Step	Expected / measure	If it fails
1. Safety check	Robot on a raised stand, wheels spin freely; no loose wires; camera has a clear view; emergency stop reachable	—
2. Wall following	Wall-following only, no pillars: robot holds a consistent distance; variation < 2 cm over a lap	Adjust `area_wall_kp` / `area_wall_kd` in `challenge_profile.py`
3. Obstacle avoidance	Green pillar in path: robot steers left then returns to the wall; clearance > 5 cm. Repeat with green and red at 20, 50, 80 cm	Adjust `GREEN_PILLAR_KP`, `RED_PILLAR_KP`, `PILLAR_STEER_OFFSET_DEG`
4. Complete run	Full state machine (wait → warm-up → laps → park): 3 laps + park in the pink lot; record time, parking accuracy, crashes	Review the recorded video (`recordings/`) frame by frame
5. Combinations	Run layouts from the Random Combination Generator in both directions to tune overall consistency	—

First test checkpoint

☐ Wall-follows 1 full lap without crashing ☐ Avoids pillars without overshooting ☐ Encoder ticks match wheel rotations ☐ Latency consistent at ~150 ms ☐ No thermal issues (PCB, driver, regulators cool)

13.5 Future Improvements

1. **Dynamic colour ranges:** instead of fixed LAB thresholds tuned on the day, measure the mat and wall brightness during camera warm-up and shift the LAB limits automatically. This would improve pillar, line and parking-block detection across many lighting conditions and remove most on-field re-tuning.
2. **Lock camera exposure and white balance after warm-up:** auto-exposure can still drift as the robot turns toward a bright window or a dark corner, which shifts colours between frames. Locking exposure and white balance once they settle at the start would keep every frame consistent for the whole run.
3. **Remember the pillar layout from lap 1:** the pillars do not move during a run, so the robot could record which side it passed each pillar on in lap 1 and plan laps 2 and 3 from that memory. Vision would only have to confirm each pillar instead of reacting to it, allowing a faster, smoother line.
4. **Main fuse on the battery output:** our converters and motor driver protect everything after them, but not the wiring between the battery and those parts. A ~5 A slow-blow fuse at the battery would cut power instantly if that wiring shorted (e.g. a pinched wire after a crash), protecting the pack and preventing a fire, without ever tripping during normal runs (2 A peak).
5. **Battery voltage monitoring:** a simple voltage divider into a Nano analog pin would let the robot report its battery level at start-up and warn us before a run on a low pack, the main remaining cause of run-to-run variation.

13.6 GitHub Utilisation: Issues & Projects

For the Nationals → Internationals phase we used GitHub as more than a place to store files. **GitHub Issues** and **GitHub Projects** kept the three of us organised and made debugging faster.

Tool	How we used it	Why it helped
Issues	Every bug or improvement got its own issue: what we saw on the mat (the symptom), the run or video it happened in, what we think caused it, and what we changed. The issue was closed once the fix was tested.	Problems were not forgotten between practice sessions, and each fix has a written record of its cause; that record is how the failure analysis in this journal was written.
Projects	A project board for the Internationals work, with each issue moved through To Do → In Progress → Done and assigned to the person working on it.	Everyone could see what was left before Internationals and who was on it, so no two people worked on the same thing and nothing was missed.
Repository	One shared repository for code, CAD, schematics, photos and documentation, with Nationals and International code kept side by side.	A judge or another team can see exactly how the robot changed between competitions and rebuild either version.

Conclusion

Our goal was never simply the fastest machine on the track. It was to design, build and validate a complete autonomous system from scratch, using the lessons of VHR Motors and The Robertos as a foundation rather than a design to repeat.

Mechanically, we moved from mismatched wheels and printed drivetrain parts to uniform SPIKE Prime wheels, a metal differential on direct drive, stainless-steel axles in one rear axle casing, and calculated 100% Ackermann steering, each decision tracing back to a failure a previous robot exposed. In **power and sensors**, two full PCB failure cycles (a 7805 thermal shutdown and a motor-driver/servo isolation failure) forced a rebuild around a buck converter and an isolated servo UBEC, which became the stable architecture of both competition robots. In **software**, a single-camera LAB pipeline held the loop to ~150 ms, fast enough for priority-based arbitration to resolve outer-wall safety, inner-wall damping, pillar avoidance and wall following into one decision every cycle. For Internationals, a modular program that detects its own challenge added pillar-aware variable speed.

The International robot meets or beats every benchmark: lap times of **24 s** in the Open Challenge and **50 s** in the Obstacle Challenge against 45 s and 90 s targets, **±1 cm** parking against ±3 cm, **75 min** runtime and a **95% success rate** over 15 consecutive laps, all for **$478.87**, under our $500 budget.

More important is what the process taught us. Nearly every major decision exists because something failed first: a scrubbing tyre, an overheating regulator, a burned-out servo, a binding gear, a camera that saw too much and too little. Between Nationals and Internationals we applied the same method to a robot that already worked: measure where it loses time and margin, find the cause, change one thing, re-test. Treating failures as data and redesigning rather than patching symptoms turned a season of iteration into a robot we trust to run consistently, not just quickly. We are proud that every subsystem was built, broken, diagnosed and rebuilt by our own hands.

We thank our coaches, mentors and the WRO community, including the teams whose documentation inspired parts of our approach, for making this season possible.

References & Technical Sources

Reference	Organisation / source	Used for
[1] WRO 2026 Future Engineers: General & Game Rules	WRO Association, 2026 season	Dimensions, weight, game requirements, obstacle randomisation procedure
[2] WRO 2026 Future Engineers: Documentation Rubric	WRO Association, 2026 season	Engineering-documentation expectations and rubric
[3] Raspberry Pi 4 Model B: Technical Specifications	Raspberry Pi	Specs, power requirements, interfaces, CSI connection
[4] Raspberry Pi Camera Module 3: Documentation & Product Brief	Raspberry Pi	Operation, resolution, IMX708 sensor, CSI-2
[5] Raspberry Pi Camera Module 3 Wide: Technical Specs	Raspberry Pi	Wide field-of-view specifications (the version we use)

Reference	Organisation / source	Used for
[6] Picamera2 Library Manual	Raspberry Pi	Camera configuration and frame capture
[7] Arduino Nano: Hardware Documentation	Arduino	Dimensions, pinout, interrupts, hardware specs
[8] DRV8871 3.6-A Brushed DC Motor Driver (PWM Control)	Texas Instruments datasheet	Voltage/current limits, PWM, protection, thermal specs
[9] L7805 Positive Voltage Regulator	STMicroelectronics datasheet	Junction temperature and thermal resistance (12.3.1)
[10] OpenCV: Colour Space Conversions	OpenCV documentation	LAB conversion and cvtColor() in the vision pipeline
[11] OpenCV: Image Processing (thresholding, morphology, contours)	OpenCV documentation	Masks, morphology, contours, track outline
[12] Ackermann Steering Geometry	X-Engineer	Inner/outer steering angles, wheelbase, track width, turning radius
[13] Team SinΘ GitHub repository	github.com/JumpyTruck/WRO-FE-2026-Sin-Theta	All CAD, STL, schematics, code and photos in this journal
[14] Team SinΘ run videos	youtu.be/3cM_T3cttLk · youtu.be/weySsihPY30	Open and Obstacle Challenge performance